BASE24®

ITS Kernel Objects Reference Manual

Print Record

07/2000, Version 3 (Reissue)
03/97, Version 2 (Reissue)
07/95, Version 1 (New)

Contents

Introduction *continued*

Event Object *continued*

Section 5

Section 6

Section 7

Section 8

Section 9

Preface

The ***BASE24 ITS Kernel Objects Reference Manual*** describes the kernel objects used in an integrated transaction services (ITS) environment. It provides an introduction to the kernel objects and the processing they perform. It also describes the requirements for configuring the kernel objects.

Audience

This manual is intended as a introductory guide for developers using the ITS environment as the basis for development of object-oriented applications as well as an informational guide for anyone who needs or desires a better understanding of kernel object processing. It does not provide any instruction on object or program development. The sole purpose of the manual is to familiarize the reader with the kernel objects and the processing they perform.

Prerequisites

Readers of this manual should be familiar with the information provided in the ***BASE24 ITS Kernel Files Maintenance Manual*** and the ***BASE24 ITS Kernel Objects Configuration Manual***. Much of the material presented in the ***BASE24 ITS Kernel Objects Reference Manual*** is founded upon the information provided in these manuals.

Prior knowledge of object-oriented programming methodology is not required but may prove helpful.

Additional Documentation

The BASE24 documentation set is arranged so that each manual presents a topic or group of related topics in detail. When one manual presents a topic that has already been covered in detail in another manual, the topic is summarized and the reader is directed to the other manual for additional information. Information has been arranged in this manner to be more efficient for readers who do not need the

additional detail and at the same time provide the source for readers who require the additional information. This manual contains references to the following BASE24 publications:

- The ***BASE24 Files and Record Structures Reference Manual*** provides instructions on how to use the DDLDOC program.
- The ***BASE24 ITS Kernel Files Maintenance Manual*** describes the ITS configuration and control environment. These facilities are used when configuring or controlling any ITS-based process.
- The ***BASE24 ITS Kernel Configuration Manual*** describes the configuration files and procedures required to configure the ITS environment.
- The ***BASE24 IEP Objects Reference Manual*** contains detailed information on integrated endpoint (IEP) objects and provides a list of the kernel objects required to build an Integrated Endpoint process.
- The ***BASE24 Logical Network Configuration File Manual*** contains detailed descriptions of Logical Network Configuration File (LCONF) assigns and params used by kernel objects.

This manual also contains references to the following NET24 publications:

- The ***NET24-XPNET Application Programming Guide*** describes the fields used in the message header of XPNET 2.1 network messages.
- The ***NET24-XPNET Network Control Operations Guide*** provides detailed procedures for starting and controlling ITS processes that run under control of the XPNET process.

This manual also contains references to the Tandem ***Guardian Procedure Errors and Messages Manual***, which describes conditions that can cause traps.

Software

This manual documents standard processing for integrated transaction services release 1.3.

Customer-specific code modifications (that is, CSMs) may result in processing that differs from the material presented in this manual. The customer is responsible for identifying and noting these changes.

Manual Summary

The following is a summary of the contents of this manual.

"Conventions Used in this Manual" follows this preface and describes notation and documentation conventions necessary to understand the information in the manual.

Section 1, "Introduction," introduces the integrated transaction services environment kernel objects. It includes the following topics:

- An introduction to each of the kernel objects and kernel object terminology.
- A discussion of how objects communicate with one another.
- A description of all kernel object member functions.
- An overview of how kernel object member functions are documented in the individual object description sections of this manual.

Section 2, "Application Network Services Object," introduces the Application Network Services object, which provides the main procedure for an object-oriented process running in the integrated transaction services (ITS) environment.

Section 3, "Class Object," introduces the Class object, which provides message handler object ID retrieval processing for new messages received by an ITS process.

Section 4, **"Event Object,"** introduces the Event object, which provides an interface to the ACI logging utilities and the Tandem Event Management Service (EMS) for the logging of ITS event messages.

Section 5, **"Log Permission Object,"** introduces the Log Permission object, which determines whether to log or suppress a particular event message at the network level.

Section 6, **"Process Control Object,"** introduces the Process Control object, which is used to interpret and dispatch network operator- or process-initiated commands to the appropriate objects.

Section 7, "Trace Object," introduces the Trace object, which provides trace facilities for the ITS processes.

Section 8, "XPNET Interface Object," introduces the XPNET Interface object which provides a communications interface between the XPNET process and an ITS process running under control of the XPNET process.

Section 9, "Processing Flows," provides examples of various scenarios that highlight the processing performed by kernel objects. These examples are presented as an aid to understanding the flow of information through the ITS environment.

A glossary that contains definitions of terms used in this manual follows the last section.

Publication Identification

Two entries appearing at the bottom of each page uniquely identify this BASE24 publication. The publication number (for example, OK-AE000-03 for the ***BASE24 ITS Kernel Objects Reference Manual*** appears on every page to assist readers in applying updates to the appropriate manual. The publication date (for example, 07/2000 for July, 2000) identifies the last time material on a particular page has been revised. This date can be used to verify that the reader has the most current information and is particularly helpful when dealing with HELP24 to answer operating questions.

The print record, located on the page immediately following the title page, is also helpful when verifying that a manual contains the most current information. An entry is added to the print record each time a BASE24 publication is modified in any way. Each print record entry includes the date of the change, a unique version number, and the type of change (new, update, reissue, or preliminary). Version 1 is assigned to a *new* manual. Later versions can be updates or reissues, depending on the changes being made. An *update* typically involves changes on a small number of pages throughout the manual. A *reissue* occurs when all pages of the manual are replaced because changes affect a large number of pages. A manual is identified as *preliminary* when it is released to customers on a limited basis, but it has not yet reached final form.

Conventions Used in this Manual

This section explains how syntax descriptions and graphic symbols are documented in this manual. It also explains a change in Nucleus process terminology.

Syntax Descriptions

Syntax descriptions in this manual use several symbols and conventions to denote how commands must be entered. These conventions are described in the table below.

Notation	Meaning
UPPERCASE LETTERS	Indicate key words and literals that must be entered exactly as shown.
lowercase italics	Identify variables that you must provide.
Brackets [ ]	Enclose optional syntax items that you can enter in the command.
Braces { }	Enclose a group of items from which you are required to choose one item. The items are separated within the braces by a vertical bar (|).
Vertical bar |	Separates alternatives in a list of items.
Ellipsis ...	Indicates the immediately preceding syntactical item can occur one or more times.
Punctuation	Must be entered as shown. This includes parentheses, commas, semicolons, and other symbols not previously described.

Spaces	Are required as delimiters wherever it would be otherwise impossible to discriminate between adjacent words or symbols. Additional spaces can be inserted between any adjacent words or symbols, but cannot be inserted within a word or multiple character symbol.
Indented lines	Indicates a continuation of the preceding line of syntax. If the syntax of a command is too long to fit on a single line, each continuation line is indented.

Graphics Symbols

In all graphics used in this manual, processes are depicted as rectangles or ovals while objects are depicted as circles.

Nucleus Process Terminology Change

There is one process that is the hub of an XPNET node. It manages message queues; controls all lines, stations, and processes in the system; and serves as an interprocess interface. Throughout this manual, this process is referred to as the XPNET process.

In other manuals, you may see references to the Nucleus process as performing these same functions. These two terms (Nucleus process and XPNET process) should be viewed as synonymous. The Nucleus process and the XPNET process perform the same functions and processing. As manuals are reissued, references to the Nucleus process will be replaced with references to the XPNET process.

Section 1

Introduction

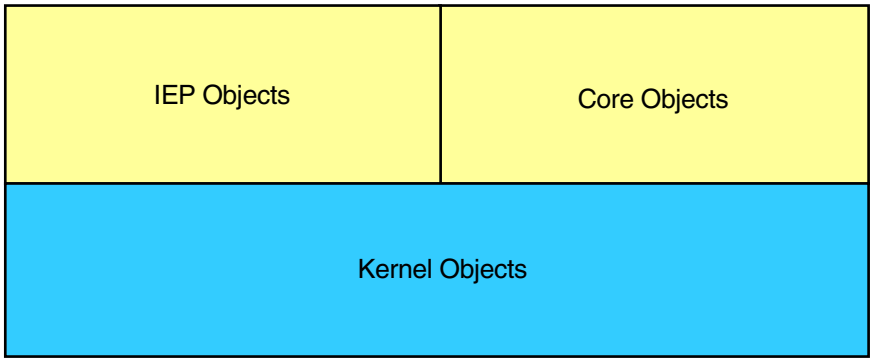
This section introduces you to the integrated transaction services (ITS) kernel objects. It covers the following topics:

- An introduction to each of the kernel objects, kernel object terminology, and how processes are built using objects
- An introduction to the extended memory tables used by kernel objects
- A discussion of how objects communicate with one another
- A library of all kernel object member functions
- A discussion on how Data Definition Language (DDL) kernel file record structures can be viewed online
- An overview of the processing flows provided in the manual

Integrated Transaction Service Objects

This subsection introduces you to each of the kernel objects and shows you how objects are used to build processes.

Most processing in the ITS environment is achieved through the use of objects (some processing in the ITS environment is still performed by procedural processes). Objects perform all the work done in object-oriented ITS processes. You can think of an object as the basic building block for an object-oriented ITS process. Some objects are considered to be kernel objects, since they provide basic services, such as event logging and process control, which are required by most applications. Other objects are termed core objects. These objects perform processing essential to an application and only reside in the application processes in which they are used. Still other objects provide functionality required specifically to interface with a particular network endpoint. These objects are referred to as integrated endpoint (IEP) objects and reside only in Integrated Endpoint processes. Integrated endpoint and core objects are built upon the foundation of kernel objects as depicted in the following illustration. Core objects and IEP objects cannot run without the basic services provided by kernel objects.



Objects are incorporated into a process by binding them into the process executable. Object binding is accomplished through the use of various MAKE files.

Kernel Objects

Kernel objects perform functions common to all object-oriented ITS processes, including process control, memory acquisition, communicating with outside entities, and logging of event messages. Each of the ITS kernel objects is described briefly below.

Application Network Services Object

The Application Network Services object contains the main procedure for an object-oriented ITS process. During process startup, the Application Network Services object directs each object in the process to initialize and sets up an environment that all objects within the process can use, including the allocation of memory. In essence, the Application Network Services object functions as the manager of an ITS process.

Currently, the Application Network Services object also contains the Space object. The Space object is responsible for allocating space in memory for objects that request it.

The Application Network Services object and the Space object are described in section 2.

Class Object

The Class object determines the object ID associated with a specific source class value for message handling purposes. When an incoming message is received by the XPNET Interface object, the Class object is called to read the Source Class Map File extended memory table to find the appropriate message handler object for the message. The Class object is described in section 3.

Event Object

The Event object provides an interface for object-oriented ITS processes to the Tandem Event Management Service (EMS) facility for logging purposes. The Event object provides a buffer and interface to EMS for the logging of informational and error events. This object is also responsible for determining whether event suppression is performed at the process level. The Event object is described in section 4.

Log Permission Object

The Log Permission object determines whether a generated event message should be logged or suppressed at the network level. The Log Permission object arbitrates all event suppression at the network level. The Log Permission object resides only in Log Permission processes and is described in section 5.

Process Control Object

The Process Control object receives network operator- and process-initiated commands from an Integrated Server Control File (ISCF) and forwards them to the appropriate object within an object-oriented ITS process. The ISCF is the mechanism by which non-network control facility commands entered by network operators, as well as commands generated by objects in other processes, are delivered to the objects within an object-oriented ITS process. The Process Control object is described in section 6.

Trace Object

The Trace object is part of the general trace facility available to all objects in the ITS environment. The Trace object uses Tandem trace utilities (that is, the Debug or Inspect product) to trace selected pieces of processing information based on the trace levels (for example, procedure level, lexicon level, etc.) configured for the object being traced. The Trace object is described in section 7.

Note: The Trace object does not support the network audit tracing that is available with procedural processes since such functionality does not apply to object-oriented ITS processes. Network audit tracing traces interactions between processes, while most interaction in the ITS environment is between objects running within the same process.

XPNET Interface Object

The XPNET Interface object receives messages through the $RECEIVE file for processes running under control of the XPNET process and dispatches them to the appropriate object in the process. The XPNET Interface object also provides the following functionality:

- Detects network overload for the process or queue
- Provides XPNET network header services to other objects

The XPNET Interface object is described in section 8 of this manual.

Integrated Endpoint Objects

Integrated endpoint objects contain the functionality needed for Integrated Endpoint processes running in the ITS environment. Integrated Endpoint processes provide the message-based or symbolic destination routing and context management functions specific to a particular endpoint. Integrated Endpoint objects reside only in the Integrated Endpoint process in which they are used; they are not used in all ITS processes. An example of an Integrated Endpoint object is the Voice Response Unit Endpoint Class Handler object for the BASE24-telebanking application. This object provides the endpoint class handling functions specific to a BASE24 Remote Banking Interface endpoint. For detailed information on Integrated Endpoint objects, refer to the ***BASE24 IEP Objects Reference Manual.***

Core Objects

Core objects contain business functionality that is required for object-oriented application processes running in the ITS environment. Typically, a core object only resides in the process where it is used. An example of a core object is any authorization object. Core objects are not described in this manual. For detailed information on core objects, refer to the application-specific transaction processing manual.

Building Object-Oriented Processes

Object-oriented processes are built by binding the appropriate kernel, core, and integrated endpoint objects into an executable process. This is done through various process MAKE files. Different objects are bound in for different processes, depending on the functionality required. In general, all kernel objects are bound into every object-oriented ITS process, except for the Log Permission object, which is only bound into Log Permission processes.

Extended Memory Tables

For performance reasons, components of the ITS environment access critical file or table data from extended memory tables held in memory rather than from disk. This saves time in the execution of time-critical operations. The usage of extended memory tables by kernel objects is discussed in the individual object description sections of this manual.

The following extended memory tables are used by kernel objects. The source files or tables from which the extended memory table data is derived, if applicable, are also provided.

Source Symbolic Map Extended Memory Table

The source symbolic map extended memory table is established from information in the Network Environment File (NEF) and is used to determine the source class for an incoming message. The source symbolic map extended memory table is a read-only hash table built by the Extended Memory Table Build utility and can be reread using the EMT WARMBOOT command. The source symbolic map extended memory table is used by the XPNET Interface object described in section 8.

Source Class Map File Extended Memory Table

The Source Class Map File extended memory table is established from the Source Class Map File (SCM) and is used to determine the message handler object for an incoming message. The Source Class Map File extended memory table is a read-only hash table built by the Extended Memory Table Build utility and can be reread using the EMT WARMBOOT command. The Source Class Map extended memory table is used by the Class object described in section 3.

Event Suppression File Extended Memory Table

The Event Suppression File extended memory table is established from the Event Suppression File (ESF) and is used for process-and network-level event suppression. The Event Suppression File extended memory table is a read-only hash table built by the Extended Memory Table Build utility and can be reread using the EMT WARMBOOT command. The Event Suppression File extended memory table is used by the Event object and the Log Permission object described in sections 4 and 5, respectively.

Process Control Extended Memory Table

The process control extended memory table is used by the Process Control object to track process control requests. The process control extended memory table is a read-write custom table that is built during initialization processing for the Process Control object; it is not built by the Extended Memory Table Build utility and cannot be reread using the EMT WARMBOOT command. The process control extended memory table is used by the Process Control object described in section 6.

Internal Tables

The ITS kernel software uses several internal tables that are built in memory during object initialization. The usage of these tables is also discussed in the individual object description sections of this manual.

Event Control Table (ECT)

The Event Control Table (ECT) is established from the Event Suppression File extended memory table and is used by the Log Permission object for network-level event suppression. The Log Permission object is described in section 5.

Object Control Block Table (OCB)

The Object Control Block Table (OCB) is used by the Application Network Services object to keep track of all the objects in a process, including each object's frame address. The Application Network Services object is described in section 2.

Object Space Table

The Object Space Table is used by the Application Network Services object to keep track of frame space allocation and deallocation on behalf of objects in the same process. The Application Network Services object is described in section 2.

Log Permission Process Table (LPT)

The Log Permission Process Table (LPT) is built from the Log Permission Process Partition File (LPF). It is used by the Event object to map subsystem IDs to specific Log Permission processes responsible for determining whether an event is logged at the network level. The Event object is described in section 4.

Process Event Timing Table (PET)

The Process Event Timing Table (PET) is used by the Event object to track the number of events to be suppressed between the logging of an event or the time interval to suppress an event before it can once again be logged. The Event object is described in section 4.

Object Communication

This subsection discusses how object-oriented ITS processes communicate at the object level and at the process level with other object-oriented and procedural processes.

Intraprocess Communication

A mechanism is required to allow for communication between the objects bound into the same process. This is accomplished through an Application Network Services object library send routine, which allows lexicon messages to be sent from one object to another object within the same process. Refer to the "Member Functions" discussion later in this section for a description of lexicons.

Interprocess Communication

The following paragraphs discuss how object-oriented processes communicate with one another and with procedural processes.

Objects in the ITS environment normally communicate with other processes (object-oriented or procedural processes) using Interprocess Communication (IPC) messages, the basic messaging mechanism used between processes. IPC messages are sent from the object-oriented ITS process through the XPNET process to their process destination(s). For messages sent to another object-oriented process, lexicons are enveloped within the IPC. For messages sent to a procedural destination, no lexicons are used.

Objects can also communicate directly with a single process using a writeread interface to the process. For example, the Event object communicates directly with the Log Permission process. This allows the Event object to wait for a response from the Log Permission process before proceeding with processing. For messages sent through the XPNET process, the sending process typically does not have to wait for a response before continuing processing.

If needed, object-oriented ITS processes can also use a writeread interface to communicate directly with a procedural process, thereby bypassing the XPNET process.

Process Control Communication

All network operator-initiated and process-initiated commands in the ITS environment are written to an Integrated Server Control File (ISCF). There is one ISCF for each server class type in the ITS environment. Each ITS server class reads a particular ISCF for its commands. All processes monitor their ISCF and detect when the end-of-file changes. When the end-of-file changes, the ISCF is read.

All object-oriented processes in the ITS environment are assigned to a server class. In addition, any procedural processes (for example, the Refresh process for the BASE24-telebanking product) that use the ISCF to receive commands are also assigned to a server class.

For object-oriented processes, each record in the ISCF indicates the PPD name of the process to receive the command, the ID and name of the object to receive the command, the name of the command to be performed, and any function data associated with the command. A PPD value of ALL indicates that all processes in the server class are to receive the command. For procedural processes, the object ID and name are not used.

Process Identification Number

XPNET release 2.1 or greater supports the extended features of the Tandem D-series operating system that enable a process to run at a high process identification number (PIN). A PIN is a unique, system-assigned identifier with a numeric value that identifies processes running in a CPU. The Tandem D-series operating system allows up to 65,535 processes to run simultaneously in a single CPU. High PIN is defined as a process identification number in the range of 256 to 65,534. Low PIN refers to a process identification number in the range of 0 to 254. PIN 255 is reserved by the system. Both the XPNET process and all object-oriented ITS processes controlled by the XPNET process can be started at either a low or high PIN.

A process runs at a high PIN when the executable object file contains the HIGHPIN file attribute, or if you start the process from TACL with the HIGHPIN ON attribute.

Note: High PIN functionality is not available if your ITS environment runs under XPNET release 2.0.

Memory Pool Usage

XPNET release 2.1 or greater supports the extended features of the Tandem D-series operating system that enable a process to use a large-memory model as well as a small-memory model. ITS processes running under control of the XPNET process can use either memory model. Using the small-memory model, ITS objects can allocate space in the upper 64K bytes (32K words) of the data stack and use message buffers of 4096 bytes. Using the large-memory model, ITS objects can allocate space within a flat segment. Flat segments allow memory space to be larger than 65,536 bytes, use message buffers of 32,000 bytes, and require a D30 or later operating system running on a Tandem NonStop Series/ RISC machine.

A large-memory model toggle set when the Application Network Services object is compiled determines whether the object allocates object memory space using the small-memory model or large-memory model. If the toggle is set and the operating system and hardware requirements are met, the large-memory model is used. Otherwise, the small-memory model is used.

Note: The large-memory model is not available if your ITS environment runs under XPNET release 2.0 or if your Tandem system is not RISC-based.

Member Functions

A member function is an action that one object can request another object to perform. Because each object is responsible for different data and for the functions related to that data, the member functions supported by each object are different. For example, any object can request that the Event object log an event message using the Log member function, however, only the Event object can perform the Log member function.

When an object asks another object to perform a function, the object making the request is considered to be the client. The client makes a request for services from another object. The object that performs the action is called the actor. The actor responds to the client's request for service. An object can function as either a client or actor, depending on what processing is being performed.

Member functions are logically grouped into lexicons. A lexicon defines all of the member functions that a client object can request of an actor object. The lexicon name identifies the client and actor objects. For example, the Process Control to Trace lexicon identifies the member functions that the Process Control object, as a client, can request of the Trace object, as an actor.

Some lexicons define functions that can be sent from any object or can be received from any object. In this case, the client or actor object is generically referred to as Object in the lexicon name. For example, the Object to Application Network Services lexicon defines the member functions that can be sent from any object, as a client, to the Application Network Services object, as an actor.

Member function messages can be nested within one another. For example, the Message member function is nested within a Message In member function by the XPNET Interface object when an Integrated Server process receives a message from an application process. The only limit as to the number of messages that can be nested within one another is the physical size of the message buffer used.

The following pages present the member functions that can be performed by kernel objects as actors. These member functions are presented according to the lexicon in which they are grouped. Lexicons are arranged in alphabetical order by lexicon name. The Version member function is presented first instead of being repeated with each lexicon because it serves the same purpose in every lexicon.

All Lexicons

The following member function is included in all lexicons.

Version — Allows for object version synchronization between the actor and client object.

The minimum actor version of the client object must be equal to or less than the supported minimum version for the actor object to perform the requested function. If not, the requested function is not performed. All the information required for this member function is provided in the lexicon header prefixed to all lexicon member function request and response messages.

Application Network Services to Object Lexicon

The following member functions are included in the Application Network Services to Object lexicon. These functions can be sent from the Application Network Services object to any object.

Initialize — Indicates that the actor object is to begin phase 1 initialization.

The Application Network Services object sends this function to all other objects, except for the XPNET Interface object, during phase 1 initialization processing. When received, the object takes whatever initialization steps are necessary to begin processing, such as obtaining space for its frame, initializing its frame, registering with the Application Network Services object, reading any Logical Network Configuration File (LCONF) assigns and params, and opening any files used for processing. After the XPNET Interface object has been initialized, this function is sent to every other object in the process, beginning with the Event object. The Event object is initialized with default data before this function is sent to the XPNET Interface object. Therefore, the Event object will have already obtained frame space, initialized its frame, and registered with the Application Network Services object when this function is received.

If the actor object does not require notification for phase 2 initialization, the object will take whatever steps are necessary to complete initialization.

Note: Version 1001 of this member function includes a memory model parameter that indicates whether the object is to obtain memory space using the small-memory or large-memory model. For more information on these models, refer to the memory pool usage discussion elsewhere in this section.

Notify — Indicates that the actor object is to begin phase 2 initialization.

The Application Network Services object sends this function to any other object that requires phase 2 initialization processing. When received, the object takes whatever actions are necessary to complete initialization. This member function indicates that phase one initialization processing is complete and that phase 2 processing can begin.

During phase 2 initialization, objects can communicate with other objects. For example, a Terminal object registers with the Common Terminal object. Objects cannot call other objects during phase 1 initialization because there is no guarantee that the object to be called has initialized yet. The exceptions to this rule are the Application Network Services, Event, Space, and XPNET Interface objects, which are always initialized before any other object in the process.

Stop ANS — Informs the actor object that the process of which it is a part is terminating.

The Application Network Services object sends this function to any other object when the process in which the objects reside is stopping. Objects indicate during registration whether or not they are to be informed when the process stops running. When received, the actor object performs any required file or extended memory table cleanup.

Guardian stop messages indicating that a spawned process has stopped are received through the System Message member function rather than through this member function.

System Message — Contains any Tandem Guardian system message received from the transport layer, except for signal timeout messages. The System Message function is a member of either the Transport Layer Object to Application Network Services lexicon or the Application Network Services to Object lexicon.

This function is sent from the XPNET Interface object to the Application Network Services object when a Guardian system message other than a signal timeout is received by the XPNET Interface object. When received, the Application Network Services object dispatches the message using a System Message member function to all objects in the process requesting system messages. Objects indicate whether they are to receive system messages during initialization processing.

System messages handled by this member function include all Guardian system messages except for Time Signal and Process Time Signal messages. Timer messages are handled through the System Time Signal member function in the Transport Layer Object to Object lexicon.

Application to Object Lexicon

The following member function is included in the Application to Object lexicon. This function can be sent from an ITS application to any object.

Message — Specifies the object ID of the destination object to receive a message received from an application process.

This function is sent from an application process to an object in an object-oriented process and is received by the XPNET Interface object. A writeread interface rather than the XPNET process is used to deliver this message. This member function contains the object ID of the destination object to receive the message. The XPNET Interface object forwards the message to the appropriate object by enveloping the member function within a Message In member function.

If a source class is specified in the Message member function, the actor object obtains the destination object for the message by calling the Class object with the Object ID Retrieve member function. If the source class is not specified in the Message member function, the destination object is specified by the destination object ID provided in the function. In either case, the actor object delivers the message to the destination object.

Event to Log Permission Lexicon

The following member function is included in the Event to Log Permission lexicon. This function can be sent from the Event object to the Log Permission object.

Event Log — Specifies the subsystem identifier and the event number of an event to be checked for suppression at the network level.

This function is sent from the Event object to the Log Permission process to retrieve network-level event suppression information. If the event is to be suppressed for a time interval, the timestamp for when the event can be logged again is provided in the response.

Object to Application Network Services Lexicon

The following member functions are included in the Object to Application Network Services lexicon. These functions can be sent from any object to the Application Network Services object.

Address Change — Indicates to the Application Network Services object that the client object has created a new frame.

This function is sent from any object to the Application Network Services object when the object has created a new frame. The Application Network Services object assigns the memory address of the frame to the object in the Object Control Block Table (OCB).

Register — Registers the client object with the Application Network Services object during phase 1 initialization, thereby making it available to communicate with other objects.

This function is sent from any object to the Application Network Services object during phase 1 initialization processing. When received, the Application Network Services object adds the object to the OCB, thereby making it available for other objects to communicate with it.

Object to Class Lexicon

The following member function is included in the Object to Class lexicon. This function can be sent from any object to the Class object.

Object ID Retrieve — Retrieves the ID of the message handler object associated with the station or process from which a message originated.

This function is sent from the XPNET Interface object to the Class object to determine the object that handles messages from a given source. The Class object uses the source class to obtain the message handler object ID from the Source Class Map File extended memory table.

Object to Event Lexicon

The following member function is included in the Object to Event lexicon. This function can be sent from any object to the Event object.

Log — Contains an event message generated by the client object.

This function is sent to the Event object from any object that has a message to be logged to EMS. The Event object will then perform one of the following actions based on settings in the Event Suppression File extended memory table and the Log Permission Process Table (LPT):

- Suppress the event
- Log the event
- Send a member function to the Log Permission process to determine whether the event is logged or suppressed at the network level

Object to Process Control Lexicon

The following member function is included in the Object to Process Control lexicon. This function can be sent from any object to the Process Control object.

ISCF Write — Indicates that a record needs to be written to an Integrated Server Control File (ISCF) on behalf of the client object.

This function is sent by any object to the Process Control object when a record needs to be written to the ISCF. A record is written to the ISCF any time an object needs to communicate with another object outside of its home process. This occurs when an object-initiated command needs to be sent to a set of processes or when a response to a request needs to be sent. Not all requests involve a response to the ISCF. Currently, only the File Close/Open response is sent to inform the Enscribe Refresh process of cutover to a new refresh file.

The originator of the request can determine which ISCFs the request needs to be written to in two different ways. If a process symbolic name is provided with the request, then the ISCF for that process' server class receives the ISCF Write member function. If only an object ID is provided, optionally with a process type, the object utilizes the existing ITS configuration files to determine the ISCF recipients of the ISCF Write member function.

The set of processes or server classes to which a command is intended is recorded in the process control extended memory table.

Object to Space Lexicon

The following member functions are included in the Object to Space lexicon. These functions can be sent from any object to the Space object. The Space object is contained in the Application Network Services object.

Memory Free — Specifies the location of memory the client object no longer needs.

This function is sent from any object to the Space object when the client object has frame space in memory it no longer needs. When received, the Space object places the space back in the memory pool to be made available again.

Memory Get — Specifies the size of space in memory needed by the client object.

This function is sent from any object to the Space object when the client object requires frame space in memory. The Space object obtains space from the memory pool and provides the memory address in the response. The response to this member function also indicates whether memory was successfully obtained.

Object to Trace Lexicon

The following member function is included in the Object to Trace lexicon. This function can be sent from any object to the Trace object.

Add Trace — Specifies trace data to be added for a trace in progress.

The function is sent from any object currently undergoing a trace to the Trace object when it has trace data to be added for a trace in progress. When received, the Trace object either adds the specified data to the trace file or displays the data directly to a trace terminal.

Object to Transport Layer Object Lexicon

The following member function is included in the Object to Transport Layer Object lexicon. This function can be sent from any object to the XPNET Interface object. Currently, the XPNET Interface object is the only transport layer object used.

Message Out — Specifies a message that needs to be delivered outside of the local Integrated Server process.

This function is sent from any object to the XPNET Interface object when a message needs to be delivered outside of the local Integrated Server process. This member function provides the following:

- The symbolic name of the process or station to which the message is to be sent
- The destination object ID
- The source object ID
- A flag indicating whether a Message to Object member function in the Transport Layer Object to Transport Layer Object lexicon needs to be added to the message
- The address and length of the message data
- The length of time, in hundredths of seconds, the message can be queued by the XPNET process

This member function is received from any object in the local process that has a message response to send in the reply to the original transaction request. The message text is sent to the originating process with a reply still pending. Only one message reply can be sent to the originating process. All additional messages are buffered to be sent to the XPNET process. All messages destined for non-object oriented processes are sent using this member function.

Object to XPNET Lexicon

The following member functions are included in the Object to XPNET lexicon. These functions can be sent from any object to the XPNET Interface object to retrieve information from a network message header or set information in a network message header. In these member functions, network refers to the XPNET network.

For detailed information on XPNET message header fields, refer to the ***NET24-XPNET Application Programming Guide***.

Header Data Retrieval Functions

The following member functions retrieve specified information from the header of incoming network messages. Because message header information is retrieved from the input buffer, these functions cannot be used until after the incoming message is read. None of these functions contain a request body.

Message Header Control Flag On — Retrieves the value of the Control Flag field from the header of an incoming network message. This field differentiates control messages from data messages.

Message Header Get Destination Queue State — Retrieves the network overload management (Network Overload Management Percent field) and queue alert threshold (Queue Alert Threshold Percent field) percentages for a symbolic destination (that is, the process that received this message).

Message Header Get Message Disposition — Retrieves the value of the Disposition field from the header of an incoming network message. This field indicates the action to take when a message cannot be delivered to its intended destination.

Message Header Get Message Failure — Retrieves the value of the Failure Code field from the header and the value of the failure type from the body of an incoming network message. This field indicates the status of the message.

Message Header Get Message Priority — Retrieves the value of the Priority field from the header of an incoming network message. This field indicates the priority assigned to the message. Messages with a higher priority are delivered to their destination before messages with a lower priority.

Message Header Get Message Process Words — Retrieves up to four application process-defined words from the header of an incoming network message.

Message Header Get Message Sequence Number — Retrieves the value of the Sequence Number field from the header of an incoming network message. A sequence number is assigned to the message by the transmitting process, which in this case, is the XPNET process since the message is incoming. On outgoing messages passed to the XPNET process for delivery, the sequence number is assigned by the application process.

Message Header Get Message Timeout — Retrieves the value of the Time Limit field from the header of an incoming network message. This field indicates the time by which the message must be delivered to its destination or be returned to its source. Application processes set this field to the number of ticks (.01-second intervals) to be added to the Timestamp field (the time the message was received by the XPNET process) in the header.

Message Header Get Message Timestamp — Retrieves the value of the Timestamp field from the header of an incoming network message. This field indicates the time the message is received by the XPNET process.

Message Header Get Message Transmission Number — Retrieves the value of the Transmission Number field from the header of an incoming network message. This field contains a system message number assigned by the XPNET process for each message.

Message Header Get Source Type — Retrieves the value of the source type from the Source Object Type field in the header of an incoming network message. This field indicates whether the message originated internally or from a process, station, or user-defined source.

Message Header Get Symbolic Destination — Retrieves the value of the Symbolic Destination field from the header of an incoming network message. This field contains the symbolic name of the ITS process that received this message.

Message Header Get Symbolic Source — Retrieves the value of the Symbolic Source field from the header of an incoming network message. This field contains the symbolic name of the process or station that originated this message.

Message Header Logical Acknowledgment Requested — Retrieves the value of the Acknowledgment Flag field from the header of an incoming network message. This field indicates whether the sender of the message requested a logical acknowledgment to be returned after this message has been delivered.

Message Header Message Force Queue On — Retrieves the value of the Force Queue field from the header of an incoming network message. This field specifies whether the message is force queued to a station or returned to the source when the station is unavailable at the time the message was delivered.

Message Header Message Function Management Present — Retrieves the value of the Function Management Flag field from the header of an incoming network message. This field indicates whether the message is a function management message generated by the XPNET process.

Message Header Message Possible Duplicate — Retrieves the value of the Possible Duplicate field from the header of an incoming network message. This field indicates whether the message was previously sent to the ITS process but an input or output error aborted the operation. If so, the message may have been partially or completely processed before the failure and should be treated as a possible duplicate.

Message Header Message Transparency On — Retrieves the value of the Transmit Transparency field from the header of an incoming network message. This field indicates whether the XPNET process transmitted the message in transparent mode. If so, the message text may contain binary data and no conversion or translation is required by the recipient.

Message Header SAF Indicator On — Retrieves the value of the Store-and-Forward Flag field from the header of an incoming network message. This field indicates whether the message should be stored for later delivery or returned to the sender if the destination on a remote XPNET node is not available at the time the message is routed.

Header Data Placement Functions

The following member functions set specified information in the header of outgoing network messages. Because message header information is set in the output buffer, these functions cannot be used until after the message has been processed. None of these functions contain a response body.

Message Header Set Control Flag — Sets the value of the Control Flag field in the header of an outgoing network message. This field differentiates level 1 (control) messages from level 0 (data) messages.

Message Header Set Logical Acknowledgment — Sets the value of the Acknowledgment Flag field in the header of an outgoing network message. This field indicates whether a logical acknowledgment is to be returned to the ITS process after the message has been delivered.

Message Header Set Message Audit — Sets the value of the Audit Flag field in the header of an outgoing network message. This field indicates whether the message is audited, if an audit is active.

Message Header Set Message Disposition — Sets the value of the Disposition field in the header of an outgoing network message. This field indicates the action to take when a message cannot be delivered to its intended destination.

Message Header Set Message Failure — Sets the value of the Failure Code field in the header of an outgoing network message. This field indicates the status of the message.

Message Header Set Message Force Queue — Sets the value of the Force Queue field in the header of an outgoing network message. This field specifies whether the message is force queued to a station or returned to the source when the station is unavailable at the time the message is delivered.

Message Header Set Message Priority — Sets the value of the Priority field in the header of an outgoing network message. This field indicates the priority assigned to the message by the ITS process. Messages with a higher priority are delivered to their destination before messages with a lower priority.

Message Header Set Message Process Words — Sets up to four ITS application process-defined words in the header of an outgoing network message.

Message Header Set Message Timeout — Sets the value of the Time Limit field in the header of an outgoing network message. This field indicates the time by which the message must be delivered to its destination or be returned to its source. This field is set to the number of ticks (.01-second intervals) to be added to the Timestamp field (the time the message was received by the XPNET process) in the header.

Message Header Set Message Transparency — Sets the value of the Transmit Transparency field in the header of an outgoing network message. This field indicates whether the XPNET process is to transmit the message in transparent mode. If so, the message text may contain binary data and no conversion or translation is required by the message recipient.

Message Header Set NOM Override — Sets the value of the Network Overload Management Override field in the header of an outgoing network message. This field indicates whether or not the message should be discarded or added to the queue when network overload management is being performed on the destination queue.

Message Header Set NOM Return — Sets the value of the Network Overload Management Return field in the header of an outgoing network message. This field indicates whether or not the message should be returned to the ITS process when network overload is detected on the destination queue.

Message Header Set SAF Indicator On — Sets the value of the Store-and-Forward Flag field in the header of an outgoing network message. This field indicates whether the message should be stored for later delivery or returned to the ITS process if the destination on a remote XPNET node is not available at the time the message is routed.

Message Header Set Symbolic Destination — Sets the value of the Symbolic Destination field in the header of an outgoing network message. This field indicates the symbolic name of the object (that is, process or station) that is to receive the message.

Process Control to Application Network Services Lexicon

The following member function is included in the Process Control to Application Network Services lexicon. This function can be sent from the Process Control object to the Application Network Services object.

Debug On — Places the Integrated Server process under control of the Tandem Debug or Inspect facility.

This function is sent from the Process Control object to the Application Network Services object when a network operator issues a command to start the Tandem Debug or Inspect facility.

Process Control to Object Lexicon

The following member functions are included in the Process Control to Object lexicon. These functions can be sent from the Process Control object to any other object.

Assign Warmboot — Causes an object to reread specified Logical Network Configuration File (LCONF) assigns.

This function is sent from the Process Control object to any object when a network operator enters an ASSIGN WARMBOOT command on that object. The Assign Warmboot response does not contain a response body. Only assigns configured in the Assign File (ASGN) are reread.

Command Message — Specifies a general command entered by a network operator using the Server Control screen.

This function is sent from the Process Control object to any object for which a general command entered from the Server Control screen by a network operator is intended. The Command Message response does not contain a response body.

EMT Warmboot — Specifies an extended memory table to be reallocated by an object.

This function is sent from the Process Control object to any other object when a network operator requests an extended memory table to be reallocated. This member function instructs the affected object to either reallocate a shared extended memory table or to rebuild the extended memory table if it is not shared. The member function contains the name of the extended memory table to be warmbooted. The response to the EMT Warmboot member function does not contain a response body.

File Close/Open — Instructs an object to open a new file and close the old file.

This function is sent from the Process Control object to any object that needs to open a new file and close the old file. This is required any time a file the object has open has been renamed or rebuilt.

LCONF Warmboot — Causes the actor object to reinitialize. During reinitialization, an object rereads its Logical Network Configuration File (LCONF) assigns and params and reallocates its extended memory tables.

This function is sent from the Process Control object to any other object when a network operator requests that the object needs to reinitialize. The member function contains the name of the LCONF from which assigns and params are to be reread. Generally, this command is completed entirely or not at all. The LCONF Warmboot response does not contain a response body.

Param Warmboot — Causes the actor object to reread specified Logical Network Configuration File (LCONF) params.

This function is sent from the Process Control object to any object that needs to reread specified LCONF params because a network operator enters a PARAM WARMBOOT command. This function can only be sent to those objects that are configured to support the PARAM WARMBOOT command. The Param Warmboot response does not contain a response body.

Status — Causes the actor object to provide object status information. The status information provided may vary from one object to another.

This function is sent from the Process Control object to any other object when a network operator issues a STATUS command on an object. The actor object receiving this member function returns the object's informational status in Event Management Service (EMS) event messages. This member function does not contain a request body.

Process Control to Trace Lexicon

The following member functions are included in the Process Control to Trace lexicon. These functions can be sent from the Process Control object to the Trace object.

Trace List — Causes the Trace object to list the current status of a trace in progress.

This function is sent from the Process Control object to the Trace object when a network operator enters a TRACE LIST command from the Trace Control screen. The trace list response information is output in an EMS event message. Neither the request or response messages for the Trace List member function contain a lexicon body.

Trace Off — Specifies that a trace currently in progress is to be stopped.

This function is sent from the Process Control object to the Trace object when a network operator stops a trace currently in progress from the Trace Control screen. This function causes all tracing for all objects for a particular process to stop. When this function is received, the Trace object sends a Stop Trace member function in the Trace to Object lexicon to every object in the process currently involved in the trace.

Trace On — Indicates that a trace is to be started.

This function is sent from the Process Control object to the Trace object when a network operator initiates a trace from the Trace Control screen. When received, the Trace object forwards a Start Trace member function to every object identified in the Trace On member function. The Trace On member function also identifies the location of the trace terminal or file and lists all the trace levels to be traced for each object involved.

Trace On Secure ITD Data — Specifies that an object is to provide trace information for secure integrated transaction data (ITD).

This function is sent from the Process Control object to the Trace object when a network operator initiates a trace of secure ITD from the Trace Control screen. When received, the Trace object forwards a Start Secure ITD member function in the Trace to Object lexicon to a security object (for example, the Transaction Security object). The Trace On Secure ITD Data member function also identifies the location of the trace terminal or file to be used for the trace.

Trace to Application Network Services Lexicon

The following member function is included in the Trace to Application Network Services lexicon. This function can be sent from the Trace object to the Application Network Services object.

Request/Response Start — Causes the Application Network Services object to begin tracing all member function request and response messages on behalf of specified objects.

This function is sent from the Trace object to the Application Network Services object to begin tracing all member function lexicon request and response messages from the objects being traced. This member function contains the number of objects undergoing a trace as well as the object ID of each object undergoing a trace.

Trace to Object Lexicon

The following member functions are included in the Trace to Object lexicon. These functions can be sent from the Trace object to any other object.

Start — Starts tracing of the actor object.

This function is sent from the Trace object to any object when a network operator initiates tracing of that object. The Start member function contains all the different trace levels supported by the object to be traced. The Start response indicates whether tracing was started successfully and does not contain a response body.

Start Secure ITD — Starts internal transaction data (ITD) tracing of the actor object.

This function is sent from the Trace object to any object when a network operator initiates secure ITD tracing of that object. The Start Secure ITD member function does not contain either a request or response body.

Stop — Stops a trace currently in progress for the actor object.

This function is sent from the Trace object to any object when a network operator stops tracing of that object. When received, the object sets all trace levels to off. The Stop Trace member function does not contain either a request or response body.

Transport Layer Object to Application Network Services Lexicon

The following member functions are included in the Transport Layer Object to Application Network Services lexicon. These functions can be sent from the XPNET Interface object to the Application Network Services object. Currently, the XPNET Interface object is the only transport layer object used.

Stop Message — Causes the Application Network Services object to stop all objects running in the local process.

This function is sent from the XPNET Interface object when a network operator stops the process to which the Application Network Services object belongs. When received, the Application Network Services object dispatches a Stop member function to all objects requiring notification. The Stop Message member function does not contain a request body.

System Message — Indicates that a system message other than a signal timeout has been received from the transport layer. The System Message function is a member of either the Transport Layer Object to Application Network Services lexicon or the Application Network Services to Object lexicon.

This function is sent from the XPNET Interface object to the Application Network Services object when a system message other than a signal timeout is received by the XPNET Interface object. When received, the Application Network Services object dispatches a System Message member function to all objects that requested to receive system messages when they registered. Objects register to receive system messages during initialization processing. System messages handled by this member function include all Guardian system messages except Time Signal and Process Time Signal. Timer messages are handled through the System Time Signal member function in the Transport Layer Object to Object lexicon.

Transport Layer Object to Object Lexicon

The following member functions are included in the Transport Layer Object to Object lexicon. These functions can be sent from the XPNET Interface object to any other object.

File I/O Completion — Indicates that a file input or output operation for the actor object has completed.

This function is sent from the XPNET Interface object to any other object to indicate that a file input or output operation for the object has completed. The object ID to which the File I/O Completion function is sent is contained in the low order word of the TAG value associated with the input or output. When received, the object takes whatever actions are appropriate.

Message Failed — Indicates that a message was sent for the actor object but was not delivered to the transport layer for some reason.

This function is sent from the XPNET Interface object to any object from which a message was sent but not delivered to the transport layer for some reason. The member function contains a code indicating the reason the message could not be delivered to its intended destination. The object that originally sent the failed message then takes whatever action is appropriate. The Message Failed member function response does not contain a response body.

Message In — Indicates that a message has been received from the transport layer that is intended for the actor object.

This function is sent from the XPNET Interface object to any object when a message is received from the transport layer and is destined for that object. All message requests received are sent to their destination object using this member function. The member function specifies the object ID of the object that sent the message and indicates whether the message was enveloped within the Message to Object member function of the Transport Layer Object to Transport Layer Object lexicon. The response to the Message In member function does not contain a response body.

System Time Signal — Indicates that a timeout has occurred for the actor object.

This function is sent from the XPNET Interface object to any object when a signal timeout is received from the transport layer. This message indicates that a timeout has occurred. This message is sent from the XPNET Interface object to any object (for example, the Context Manager object) that needs to know that a timeout has occurred.

Only objects that set timers using the Tandem SIGNALPROCESSTIMEOUT or SIGNALTIMEOUT procedures can receive the System Time Signal member function.

Transport Layer Object to Transport Layer Object Lexicon

The following member function is included in the Transport Layer Object to Transport Layer Object lexicon. This function can be sent from the XPNET Interface object to the XPNET Interface object in another ITS process. Currently, the XPNET Interface object is the only transport layer object used.

Message to Object — Indicates that a message has been received from another object-oriented ITS process.

This function is used for the XPNET Interface object in one ITS process to communicate with the XPNET Interface object in another process. All messages sent from an object-oriented ITS process to another object-oriented ITS process are enveloped within a Message to Object member function. This member function specifies the object ID of the object to receive the message in the destination process as well as the object ID of the object from which the message was sent. If the message cannot be delivered, it is returned to the source object as a failed message. The receiving XPNET Interface object delivers all Message to Object member functions to the destination object in the local process.

Kernel File Documentation

Kernel file record structures are documented online using comments in the Data Definition Language (DDL) source schema files that define these structures. Kernel file record structure DDLs are located on the OK13DICT subvolume.

To improve the readability and usability of the online information, ACI provides a utility program called DDLDOC. The DDLDOC utility retrieves information from the compiled data dictionary and formats the information as the user requests. The information provided by the DDLDOC utility includes the position of the field in the file, the field name, and the data type of the field. Field names can be shown in COBOL or TAL format. Detailed field descriptions, when requested by the user, are printed below the field name and data type information.

The table below identifies the names of kernel file record structures.

File Name	DDL Record Name
Assign File (ASGN)	ASGN
Assign/Object Relationship File (AORF)	AORF
Assign/Process Type Relationship File (APRF)	APRF
EMT/Object Relationship File (EORF)	EORF
EMT/Process Type Relationship File (EPRF)	EPRF
Event Suppression File (ESF)	ESF
Extended Memory Table File (EMT)	EMT
Integrated Server Control File (ISCF)	ISCF
Logical Network Configuration File (LCONF)	LCONF
Log Permission Process Partition File (LPF)	LPF
Object File (OBJ)	OBJ
Process File (PRO)	PRO
Process/Object Relationship File (PORF)	PORF
Process Type File (PTF)	PTF

File Name	DDL Record Name
Server Class/Process Relationship File (SPRF)	SPRF
Server Class Type File (SCT)	SCT
Source Class Map File (SCM)	SCM
Source symbolic map extended memory table*	SSM

* This structure is built from the Network Environment File (NEF).

The OK13DICT subvolume also contains structures for the lexicon message header (LEX-HDR) and the request and response structures for member functions in the Process Control to Object and Process Control to Trace lexicons. For these lexicon structure names, refer to section 6.

For more information on using DDLDOC, refer to the ***BASE24 Files and Record Structures Reference Manual.***

Processing Flows

Sample processing flows are provided for a better understanding of the functions performed by kernel objects in the course of transaction processing. The following processing scenarios are depicted in section 9. These scenarios highlight the processing performed by kernel objects.

- Object ID retrieval processing
- Network operator command processing
- Starting and stopping trace
- Logging of event messages
- Application initialization

ACI Worldwide Inc.

Section 2

Application Network Services Object

The Application Network Services object provides the main procedure for an object-oriented process running in the integrated transaction services (ITS) environment. This section describes the Application Network Services object and includes the following topics:

- An overview of how the Application Network Services object fits into message processing
- A discussion of the initialization processing performed by the Application Network Services object
- A discussion of the normal processing performed by the Application Network Services object
- A discussion of how the Application Network Services object enables communication among objects running within the same process
- A discussion of the configuration requirements for the Application Network Services object
- A list of member functions supported by the Application Network Services object

Overview

The Application Network Services object provides the main procedure for an object-oriented ITS process. During process startup, the Application Network Services object calls all other objects in the process to initialize. All objects registered to the process are recorded in an internal table. During initialization, the Application Network Services object sets up an environment that all objects within the process can use, including all memory allocation for data accessed in processor memory rather than from disk. The full range of initialization steps performed by the Application Network Services object is provided under the "Initialization Processing" heading later in this section. After initialization, the Application Network Services object passes control to the XPNET Interface object, which handles application messages from the appropriate transport layer.

Currently, the Application Network Services object also provides the functionality for the Space object. The Application Network Services object allocates space in a memory pool in processor memory for objects in the same process to set up private data and retains the address of each object's frame. Objects send member functions to the Application Network Services object to obtain and release space as needed.

In essence, the Application Network Services object functions as the manager for all objects in the process. It sets up the resources (for example, memory space, interobject communication) that the objects need to accomplish their specific tasks and also dispatches any system messages and stop notification messages to those objects that require them. When the transport layer receives a system message, the Application Network Services object sends the messages to objects that requested to receive them when they initialized.

Events generated by the Application Network Services object are always sent to the Event object for logging.

When an operator enters a command to place an object-oriented ITS process under control of the Tandem Inspect or Debug product, the Process Control object sends the Debug On member function to the Application Network Services object. When the Application Network Services object receives this member function, it passes control of the process to the Tandem Inspect or Debug product.

When an operator enters a STATUS command to determine the amount of space available in memory, the Process Control object sends the Status member function to the Application Network Services object. When the Application Network Services object receives this member function, it returns object status information that is output in an event message to EMS.

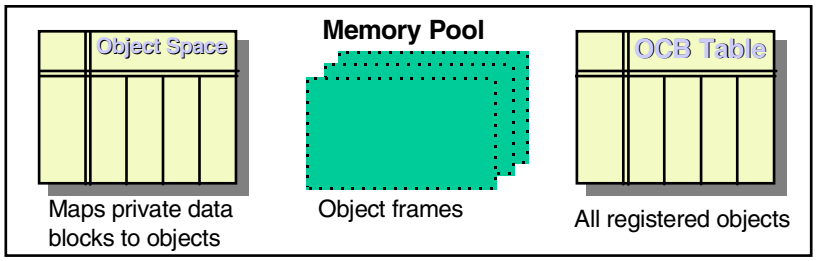
All interobject communication within an ITS process is provided by the Application Network Services procedure named ANS_MSG_SEND. In this respect, the Application Network Services object also functions as a delivery service by delivering all interobject communication messages within the process. The object maintains the address of each object's frame in an internal table.

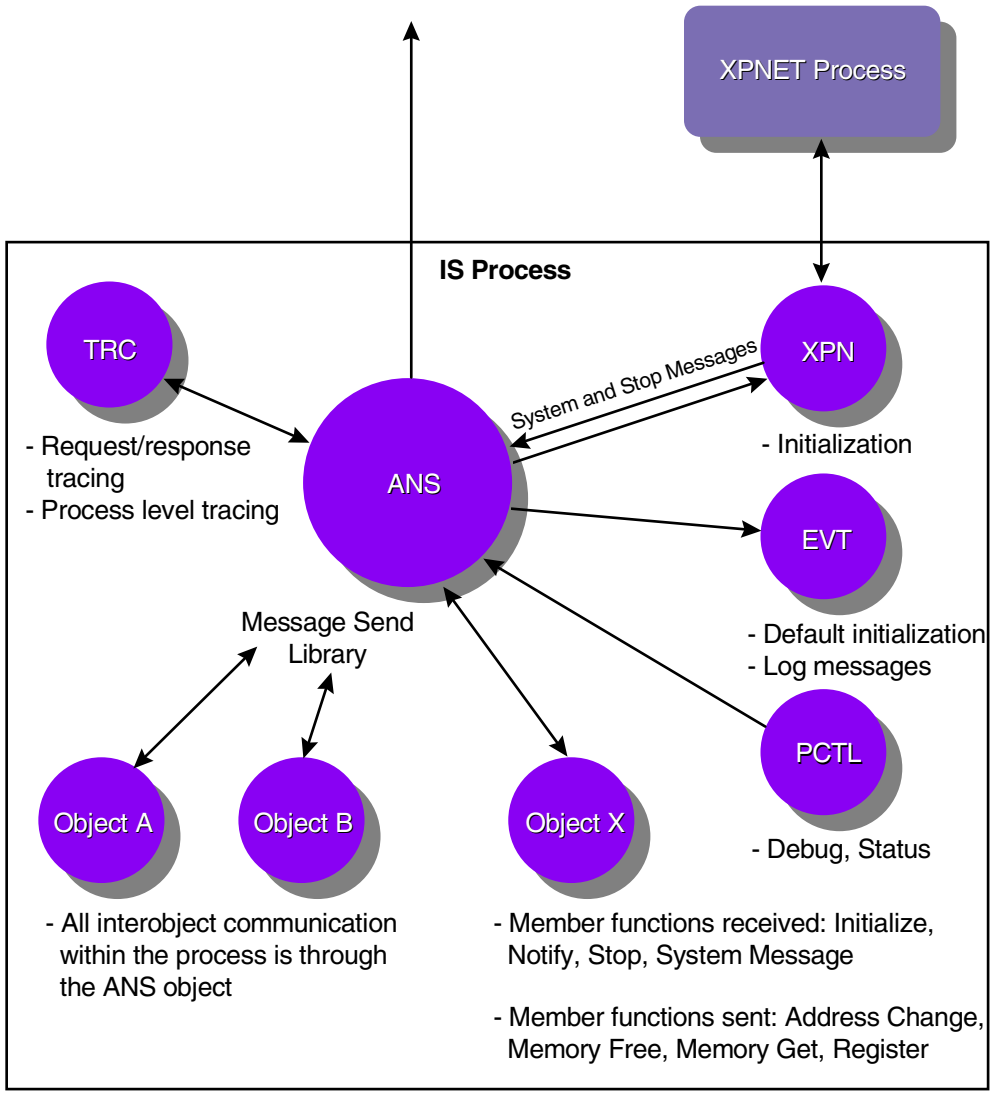
The Application Network Services object must be bound into every object-oriented process running in the ITS environment.

The functions of the Application Network Services object are depicted in the illustration on the following page. A key to the abbreviations is shown below.

Key

ANS = Application Network Services object
EVT = Event object
IS = Integrated Server
OCB = Object Control Block Table
PCTL = Process Control object
TRC = Trace object
XPN = XPNET Interface object


Memory Pool
Object Space
OCB Table
Maps private data blocks to objects
Object frames
All registered objects


XPNET Process
IS Process
TRC
- Request/response tracing
- Process level tracing
ANS
System and Stop Messages
XPN
- Initialization
EVT
- Default initialization
- Log messages
PCTL
- Debug, Status
Message Send Library
Object A
Object B
Object X
- All interobject communication within the process is through the ANS object
- Member functions received: Initialize, Notify, Stop, System Message
- Member functions sent: Address Change, Memory Free, Memory Get, Register

Initialization Processing

The Application Network Services object is the first object in a process to initialize and is responsible for setting up the processing environment for all other objects within the process. During initialization processing, the Application Network Services object performs the following tasks:

1. Initializes trap handling for the process. All traps cause the process to abend rather than being brought up under Inspect or Debug. Trap conditions are described in the ***Guardian Procedure Errors and Messages Manual***.
2. Determines whether flat segments are supported. The processor type must be a Tandem NonStop Series/RISC machine running a D30 or later operating system to support flat segments. If flat segments are supported, the large-memory model can be used. If flat segments are not supported, only the small-memory model can be used. The large-memory model toggle must be set before the large-memory model is
3. Allocates the memory pool needed for the process, as follows. All memory usage is recorded in the Object Space Table, which maps private data blocks to individual objects.
   - If the large-memory model is used, the object allocates a flat segment.
   - If the small-memory model is used, the object allocates the upper 32K of the stack.
4. Creates the Object Control Block Table (OCB). This table contains an entry for each object bound into the process. Each entry specifies the object's frame address as well as other flags used in processing.
5. Uses the EVT_INIT_DFLT routine to initialize the Event object with the default EMS collector and configuration. This allows events to be logged throughout the remainder of initialization processing.
6. Uses the TLO_INIT routine to initialize the XPNET Interface object. With the exception of the default initialization of the Event object, the XPNET Interface object must be initialized before any other object. This is so the XPNET Interface object can read any startup messages received in the $RECEIVE file and provide the necessary information for all other objects to initialize, such as the name of the Logical Network Configuration File (LCONF) and the symbolic name of the process.
7. Calls the Event object to initialize using the Initialize member function. This member function calls phase 1 initialization.

8. Calls all other objects that are part of the process to perform phase 1 initialization using the Initialize member function. After the XPNET Interface object and Event objects have been initialized, the remaining objects in the process can be initialized in any order.

   For each object the Application Network Services object initializes, the Application Network Services object performs the following:

   a. When the Application Network Services object (Space object) is called by the initializing object with a Memory Get member function, the Application Network Services object allocates frame space for the object. This space is allocated either in a flat segment or the upper 32K of the stack and is used in place of global memory. Object space usually contains the object status, trace levels, extended memory segment IDs, file names and numbers, and pointers to any other dynamically allocated data areas.

      The frame address itself is stored by the Application Network Services object on behalf of the object and is passed to the object when the object is called. Only one frame address can be registered with the Application Network Services object for each object initialized.

      A secondary frame can be allocated if space is needed for a large global buffer or a dynamic table. The address of the secondary frame is stored in the primary frame.

   b. Assigns a slot ID for each object bound into the process when an object registers by sending a Register member function to the Application Network Services object. The Application Network Services object then stores the primary frame address for the object in the OCB. At this point, other objects that have already registered can now communicate with the registered object.

9. Notifies those objects requiring notification that phase 1 initialization is complete. The Application Network Services object begins phase 2 initialization for those objects that requested it during phase 1 initialization.

   During phase 2 initialization, an object takes whatever steps are necessary to complete initialization, such as communicating with other objects. For example, a Terminal object (a core object) registers with the Common Terminal object (another core object). Objects cannot call other objects during phase 1 initialization, because there is no guarantee that the object to be called has initialized. The Application Network Services object does not begin phase 2 initialization if any object indicates that it is not ready or that a system error occurred during phase 1 initialization.

10. Calls the XPNET Interface object to start waiting for messages to process from the transport layer.

Normal Processing

During normal processing, the Application Network Services object performs the following functions in response to various member functions received from other objects in the process:

- Sets the request and response trace levels for a specified object when the Trace member function is received. Performs request and response tracing as requested. Starts and stops object request and response tracing as requested.
- Allocates and releases object memory as requested in Memory Free and Memory Get member functions received. This does not include space for extended memory tables.
- Maintains the current frame address for each object within the process using Register and Address Change member functions.
- Dispatches all system messages using System Message member functions.
- Handles all interobject communication as discussed on the following page.

Interobject Communication

All interobject communication within a process is handled through the ANS_MSG_SEND library procedure.

The Application Network Services object checks whether the object IDs of the message sender and recipient are found in the OCB. The slot ID identifies the slot the object is bound into when objects are physically bound into a process. The slot ID is set when an object registers with the Application Network Services object using the Register member function. The request message from the sender object is forwarded along with the sending object's primary frame address to the receiving object.

Configuration

This subsection lists the configuration requirements for the Application Network Services object.

Toggle Configuration

To use the large-memory model, a large-memory model toggle must be set to on (using the SETTOG directive) when the Application Network Services object is compiled. If this toggle is not set, or if flat segments are not supported, the small-memory model is assumed.

Kernel Configuration

The Application Network Services object must be configured to every process running in the ITS environment.

A record must be added to the Object File (OBJ) for the Application Network Services object. Once this OBJ record has been established, records associated with the object must be added to the Process/Object Relationship File (PORF). For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

LCONF Assigns and Params

The Application Network Services object currently does not use any Logical Network Configuration File (LCONF) assigns or params in its processing.

Member Functions

This subsection documents the member functions sent and received by the Application Network Services object.

Member Functions Sent

The table below shows the member functions sent by the Application Network Services object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services to Object	Initialize
	Notify
	Stop ANS
	System Message
Object to Event	Log
Object to Trace	Add Trace

Member Functions Received

The table below shows the member functions received by the Application Network Services object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Object to Application Network Services	Address Change
	Register

Lexicon Name	Member Function
Object to Space	Memory Free
	Memory Get
Process Control to Application Network Services	Debug On
Process Control to Object	Status
Trace to Application Network Services	Request/Response Start
Trace to Object	Start Trace
	Stop Trace
Transport Layer Object to Application Network Services	Stop Message
	System Message

ACI Worldwide Inc.

Section 3

Class Object

The Class object determines the message handler object for an incoming message. This section describes the Class object and includes the following topics:

- An overview of how the Class object fits into message processing
- A detailed discussion of the information used to determine the message handler object for a message (object ID retrieval processing)
- A discussion of the configuration requirements for the Class object
- A list of the member functions supported by the Class object

Class Object Overview

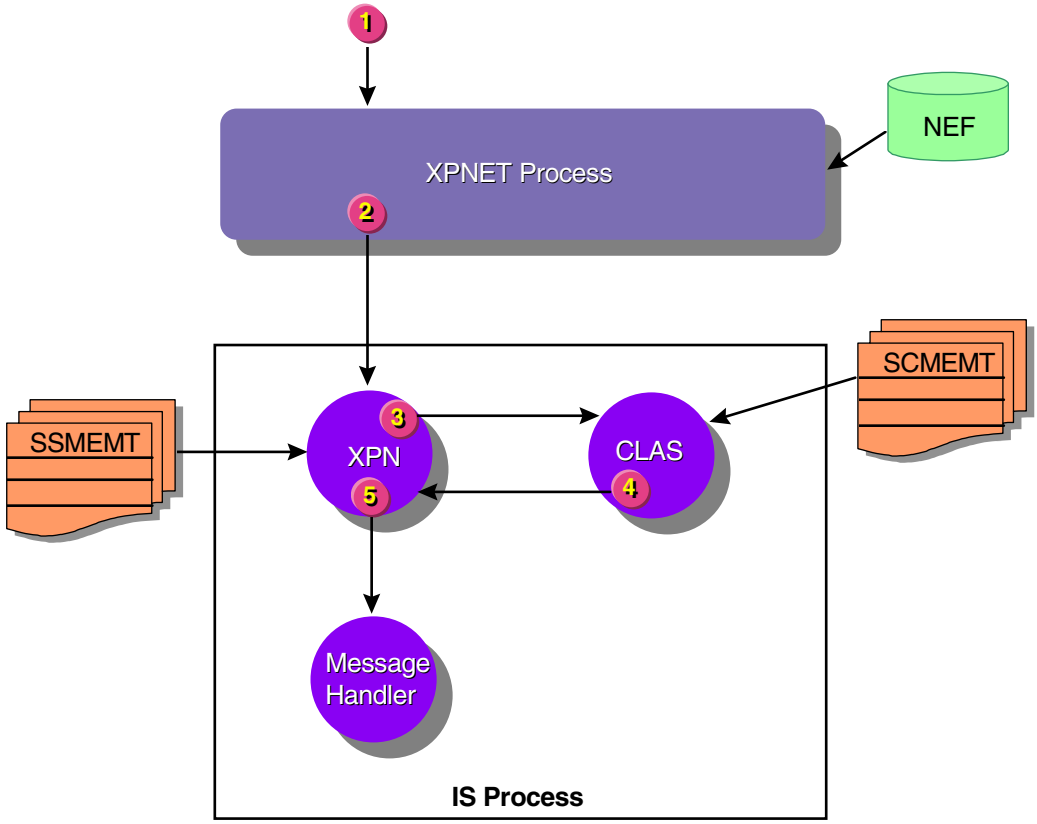
When an integrated transaction services (ITS) process receives a message, the Class object is called by the XPNET Interface object within the ITS process to determine the message handler object ID associated with the source class of the message. The source class is defined for each station or process in the source symbolic map extended memory table accessed by the XPNET Interface object or is specified directly in the message. If the destination object ID is provided in the message itself, the transport layer does not call the Class object.

When an incoming message is read by an ITS process, the XPNET Interface object determines the source class of the message. It then calls the Class object with the Object ID Retrieve member function. The Class object then reads the in-memory Source Class Map File extended memory table, built by the Extended Memory Table Build utility, to find the appropriate message handler object for the message. In this respect, the Class object functions as a router of incoming messages. Depending upon the message source, the message is typically passed on to one of the following types of objects for further processing:

- Interface stations are mapped to the interface-specific Endpoint Class Handler object
- Terminal stations are mapped to the terminal-specific Endpoint Class Handler object
- Object-oriented processes (for example, Integrated Endpoint, Integrated Authorization Server, and Interface Master processes) are mapped to the XPNET Interface object
- Procedural processes are mapped to the Internal Message Translate Interface object

The Class object must be bound in to every object-oriented process running in the ITS environment.

The functions of the Class object are depicted in the illustration on the following page. A key to the abbreviations follows the illustration. Processing steps depicted in the illustration are provided following the key.



Key

CLAS	=	Class object
IS	=	Integrated Server
NEF	=	Network Environment File
SCMEMT	=	Source Class Map File extended memory table
SSMEMT	=	Source symbolic map extended memory table
XPN	=	XPNET Interface object

The processing steps illustrated above are as follows:

1. A message source (process or station) acquires or initiates a transaction and sends it to the XPNET process.
2. The XPNET process forwards the message based on the DESTINATION attribute in the definition for the message source. In this example, the message destination is an Integrated Server process.
3. The XPNET Interface object within the Integrated Server process receives the incoming message and determines the source class from the source symbolic map extended memory table or the message itself. If the destination object ID

is specified in the message itself, the XPNET Interface object skips to step 5. Otherwise, the XPNET Interface object calls the Class object with the Object ID Retrieve member function to identify the message handler object associated with the message source.

The SOURCECLASS attribute configured for a process or station in the XPNET system is used as the source class.

4. The Class object looks up the source class from the message in the Source Class Map File extended memory table using the source class of the message as the key. The source class identifies the type of message handler object required to process the message. This is the object that determines the message class for the given message from a particular source or translates the message into Internal Transaction Data (ITD) format. The Class object returns the object ID of the message handler object to the XPNET Interface object in the Object ID Retrieve member function response.
5. The XPNET Interface object passes the message to the appropriate message handler object based on the object ID returned by the Class object or contained within the message itself.

Other processing occurs with the message; however, the Class object does not have a role in that processing.

Configuration

This subsection lists the configuration requirements for the Class object.

Kernel Configuration

The Class object must be configured to every process running in the ITS environment.

A record must be added to the Object File (OBJ) for the Class object. Once this OBJ record has been established, records associated with the object must be added to the following files:

- Process/Object Relationship File (PORF)
- EMT/Object Relationship File (EORF)

For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

LCONF Assigns and Params

The Class object uses the SCMEMT assign from the Logical Network Configuration File (LCONF) assign in its processing. For a description of this assign and for instructions on adding it to the LCONF, refer to the ***BASE24 Logical Network Configuration File Manual***.

Member Functions

This subsection documents the member functions sent and received by the Class object.

Member Functions Sent

The table below shows the member functions sent by the Class object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Object to Application Network Services	Register
Object to Event	Log
Object to Space	Memory Get
Object to Trace	Add (Trace)

Member Functions Received

The table below shows the member functions received by the Class object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services to Object	Initialize
	Stop ANS
Object to Class	Object ID Retrieve

Lexicon Name	Member Function
Process Control to Object	EMT Warmboot
	LCONF Warmboot
Trace to Object	Start Trace
	Stop Trace

ACI Worldwide Inc.

Section 4

Event Object

The Event object provides an interface for an integrated transaction services (ITS) process to the Tandem Event Management Service (EMS) facility. This section describes the Event object and includes the following topics:

- An overview of how the Event object fits into message processing
- A brief discussion on the initialization processing performed by the Event object
- A detailed discussion of how the Event object cooperates with the ACI event log utilities to log events from ITS processes, including a detailed discussion of how the Event object supports event suppression
- A discussion of the configuration requirements for the Event object
- A list of member functions supported by the Event object

Overview

The Event object maintains the data structures needed to use the ACI event log utilities and cooperates with these utilities in maintaining information about the status of paths to EMS collectors and in reestablishing these paths after transient errors. All event messages generated by ITS objects are sent to the Event object, which determines whether the event is to be logged or suppressed. For events suppressed at the process level, the object uses the Event Suppression File extended memory table to determine whether the event is logged or suppressed. For events suppressed at the network level, the object sends a member function to the Log Permission process to determine whether the event is logged or suppressed.

For those events to be logged, the object utilizes the ACI event logging utilities to write the events to the EMS log. Thus, the Event object can be thought of as the logger of event messages for an object-oriented ITS process.

For events suppressed at the network level, the Event object uses the Event Suppression File extended memory table and the Log Permission Process Table (LPT) to determine which log permission processes to open and request permission from.

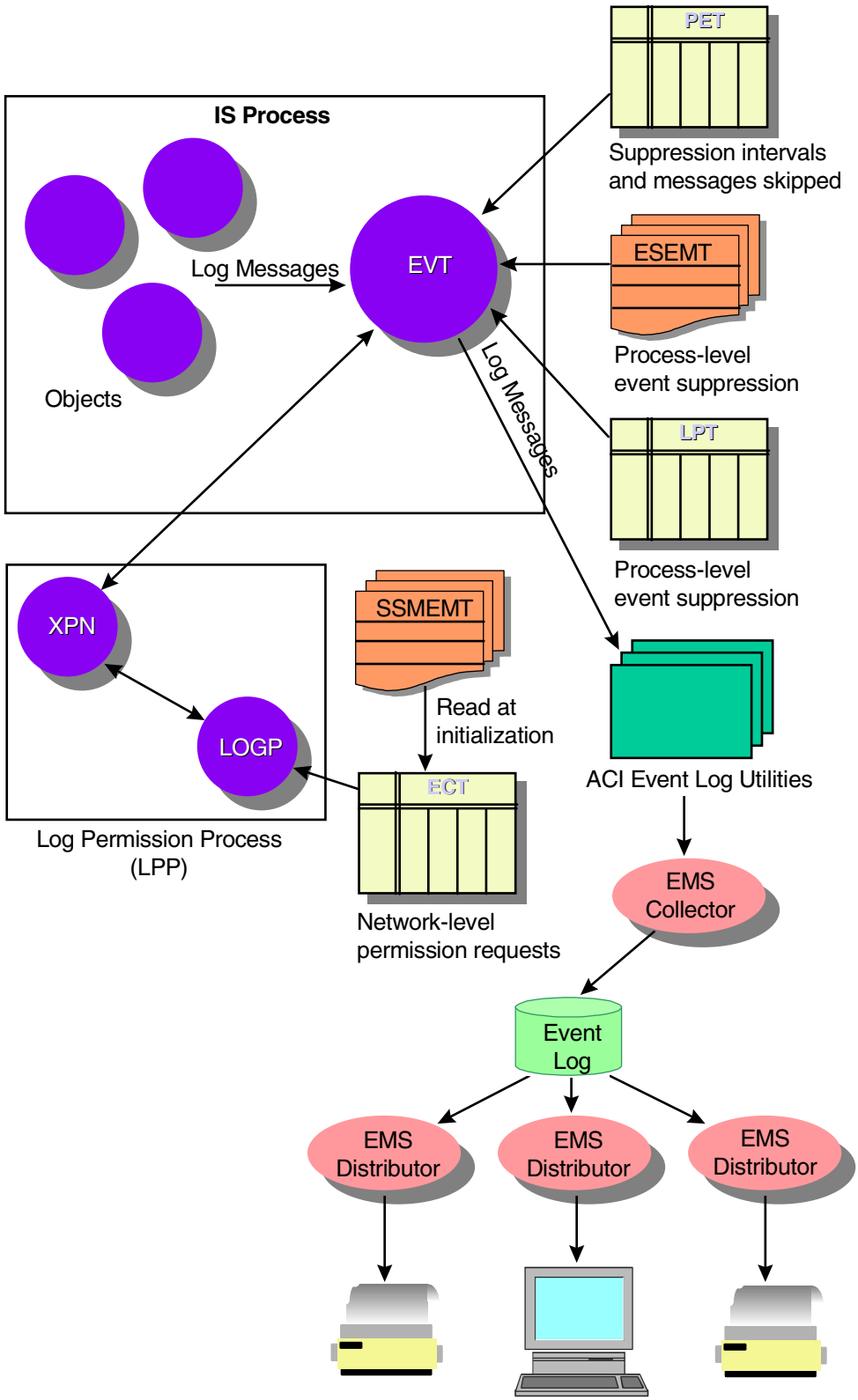
Detailed information on the configuration of event suppression is provided in the ***BASE24 ITS Kernel Configuration Manual***.

The Event object must be bound in to every object-oriented process running in the ITS environment.

The functions of the Event object are depicted in the illustration on the following page. A key to the abbreviations is shown below.

Key

ECT = Event Control Table
EMS = Event Management Service
EVT = Event object
ESEMT = Event Suppression File extended memory table
IS = Integrated Server
LOGP = Log Permission object
LPP = Log Permission Process
LPT = Log Permission Process Table
PET = Process Event Timing Table
XPN = XPNET Interface object


PET
Suppression intervals
and messages skipped
IS Process
Log Messages
EVT
ESEMT
Process-level
event suppression
Objects
Log Messages
LPT
Process-level
event suppression
XPN
SSMEMT
LOGP
Read at
initialization
ACI Event Log Utilities
ECT
Log Permission Process
(LPP)
EMS
Collector
Network-level
permission requests
Event
Log
EMS
Distributor
EMS
Distributor
EMS
Distributor

Initialization Processing

The Event object performs two types of initialization processing—default and phase 1 initialization. Default initialization is initiated by the Application Network Services object before it initializes the XPNET Interface object. This allows events generated when the XPNET Interface object initializes to be logged before the Event object executes phase 1 initialization. Both of these initialization routines are described below.

Default Initialization

The Event object performs the following default initialization processing when the EVT_INIT_DFLT routine is called by the Application Network Services. This processing allows event messages generated while the XPNET Interface object initializes to be logged before phase 1 initialization of the Event object takes place.

When default initialization is called, the Event object performs the following:

1. Sets the process symbolic name to spaces.
2. Allocates frame space.
3. Initializes the frame.
4. Registers its frame with the Application Network Services object.
5. Calls the ACI event log utility EVENT_LOG_INIT, which initializes the event buffer control block and opens the default primary EMS Collector process.
6. If no errors occurred, the object returns a value of true to the Application Network Services object.

Phase 1 Initialization

The Event object performs the following processing steps during phase 1 initialization. Phase 1 initialization is called when the Application Network Services object sends the Initialize member function to the Event object.

1. Retrieves Logical Network Configuration File (LCONF) assigns and params.
2. Warmboots event logging with the specified configuration.

3. Allocates the Event Suppression File extended memory table. This table is built from the Event Suppression File (ESF) by the Extended Memory Table Build utility.
4. Calculates the secondary frame space needed for the Log Permission Process Table by multiplying the size of an entry in the table by the value of the MAX-LPP-PROCESSES param.
5. Requests secondary frame space for the Log Permission Process Table by sending a Memory Get member function to the Space object (that is, the Application Network Services object).
6. Opens the Log Permission Process Partition File (LPF) and builds the Log Permission Process Table. This table maps subsystem IDs (SSIDs) to specific Log Permission processes responsible for determining whether an event is to be logged.
7. Reads the LCONF for LPP-*process* assigns and performs the following. These assigns identify the process pair directory (PPD) names of the Log Permission processes.
    a. Adds an entry for each assign to the Log Permission Process Table.
    b. Attempts to open each PPD specified.
8. Allocates space for the Process Event Timing Table (PET) if the value of the PROCESS-EVENT-TABLE-SIZE param is greater than 0. The object requests space for the table by sending a Memory Get member function to the Space object.
9. Initializes the Process Event Timing Table (PET). This table is used to track the number of events to suppress between the logging of an event and the time interval to suppress an event before it can once again be logged.
10. Responds to the Initialize member function.

Event Logging

The Event object performs the following processing when it receives the Log member function from an object in the ITS process.

1. Retrieves the Event Suppression File extended memory table entry using the subsystem ID and event number of the event. If an entry for the event is not found, the event is not suppressed. Processing skips to step 4.
2. Determines the log suppression level. The Event object checks the log state from the Event Suppression File extended memory table entry to determine which log suppression level is to be used.
3. Determines whether to suppress the event. The processing performed to determine whether to suppress the event varies based on the log suppression level and method, as described below:
   - If log suppression is performed at the network level (that is, event suppression is determined by the Log Permission process) and the log suppression method is based entirely or in part on the number of events, the Event object sends an Event Log member function to the Log Permission process to determine whether to suppress the event. The object obtains the name of the Log Permission process to which the member function is sent from the Log Permission Process Table (LPT).
   - If log suppression is performed at the network level and the log suppression method is based on a time interval only, the Event object checks its Process Event Timing Table (PET) to determine whether an entry exists for the event, as follows:
     - If an entry is located, the Event object checks whether the current time falls within the time interval during which the message is to be suppressed. If the current time falls within the suppression time period, the event is suppressed and no further processing occurs. If the current time is outside the suppression period, the Event object sends an Event Log member function to the Log Permission process retrieved from the LPT to determine whether to suppress the event.
     - If an entry is not located, the Event object sends an Event Log member function to the Log Permission process retrieved from the LPT to determine whether to suppress the event.

- If the log suppression level is process, the Event object checks its Process Event Timing Table (PET) to determine whether an entry exists for the event, as follows:
  - If an entry is located, the Event object checks the number of messages to skip and the event suppression skip interval against the current time. If the remaining number of messages to skip is not 0 or if the current time falls within the suppression time period, the event is suppressed and no further processing occurs. Otherwise, the message is to be logged and the PET entry is updated.
  - If an entry is not found in the PET, the event is to be logged. An entry is then added to the PET for the event.
- If the event suppression level is set to always suppress the event, the event should not be logged. No further processing occurs.
- If the event suppression level is set to always log the event, the event is always logged. In this case, processing continues with step 4.

4. If the event is to be logged, the object resets the event buffer and builds the event in the buffer.
5. Once the event buffer is built, the object calls the ACI event logging utilities to log the event to the primary EMS Collector. If the primary collector is not available, the event is logged to the alternate EMS Collector.
6. Finally, the Event object responds to the calling ITS object, indicating the outcome of the event log request. If the event is successfully suppressed, the Event object returns a no error response status, which is the same response returned if the event is successfully logged.

Configuration

This subsection lists the configuration requirements for the Event object. For detailed instructions on configuring the kernel environment and event suppression, refer to the ***BASE24 Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

Kernel Configuration

The Event object must be configured to every process running in the ITS environment.

A record must be added to the Object File (OBJ) for the Event object. Once this OBJ record has been established, records associated with the object must be added to the following files:

- Assign/Object Relationship File (AORF)
- Process/Object Relationship File (PORF)
- EMT/Object Relationship File (EORF)

Event Suppression Configuration

A record must be added to the Event Suppression File (ESF) for each event to be suppressed in some way. If network level suppression is to be used, a record must be configured to the Log Permission Process Partition File (LPF) for each range of subsystem IDs (SSIDs) used.

The LPF maps ranges of subsystem identifiers (SSIDs) to specific Log Permission processes responsible for arbitration when determining whether an event is to be logged.

LCONF Assigns and Params

The Event object uses a number of Logical Network Configuration File (LCONF) assigns and params in its processing. These assigns and params are identified below. For descriptions of these assigns and params and for instructions on adding them to the LCONF, refer to the ***BASE24 Logical Network Configuration File Manual***.

Assigns	**Params**
ALT-COLL	ALT-TIM-LIMIT
ESEMT	CONSOLE-PRT
LPF	LPP-TIME-LIMIT
LPP-*process*	MAX-LPP-PROCESSES
PRI-COLL	PRI-TIM-LMT
	PROCESS-EVENT-TABLE-SIZE

The LPP-*process* assign can be reread using the ASSIGN WARMBOOT command.

The ALT-TIM-LIMIT, CONSOLE-PRT, and LPP-TIME-LIMIT params can be reread using the PARAM WARMBOOT command.

Member Functions

This subsection documents the member functions sent and received by the Event object.

Member Functions Sent

The table below shows the member functions sent by the Event object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Application to Object	Message
Object to Application Network Services Object	Register
Object to Space	Memory Free
	Memory Get
Object to Trace	Add Trace
Event to Log Permission Object	Event Log

Member Functions Received

The table below shows the member functions sent by the Event object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services Object to Object	Initialize
	Stop ANS

Lexicon Name	Member Function
Object to Event	Log
Process Control to Object	Assign Warmboot
	EMT Warmboot
	LCONF Warmboot
	Param Warmboot
Trace to Object	Start Trace
	Stop Trace

ACI Worldwide Inc.

Section 5

Log Permission Object

The Log Permission object determines whether to log a particular event message or suppress it at the network level. This section describes the Log Permission object and includes the following topics:

- An overview of how the Log Permission object fits into message processing
- A detailed discussion of how the Log Permission object supports network-level event suppression
- A discussion of the configuration requirements for the Log Permission object
- A discussion of the operator-initiated general server commands that can be sent to the Log Permission object
- A list of member functions supported by the Log Permission object

Overview

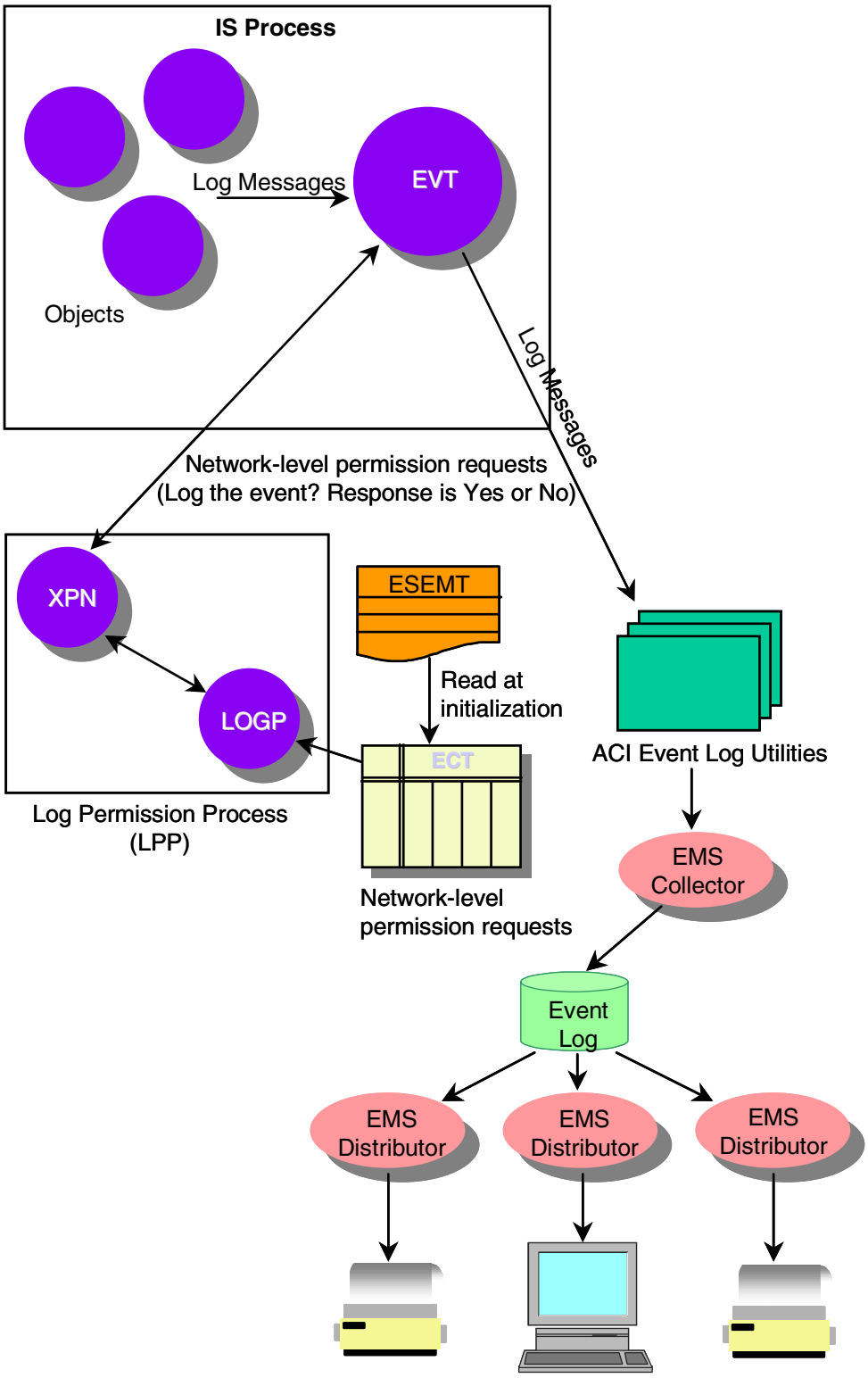
The Log Permission object determines whether it is permissible to log a particular event message or whether that event message is to be suppressed at the network level. The Log Permission object maintains information about all events that have been suppressed at the network level. Each Log Permission process performs network-level event suppression for a particular range of subsystem identifiers. A unique subsystem identifier is assigned to each object in the ITS environment.

The Log Permission object is bound in and configured only within Log Permission processes running in the integrated transaction services (ITS) environment.

The functions of the Log Permission object are depicted in the illustration on the following page. All network-level suppression requests are handled by the Log Permission object. A key to the abbreviations is shown below.

Key

ECT	=	Event Control Table
EMS	=	Event Management Service
EVT	=	Event object
ESEMT	=	Event Suppression File extended memory table
IS	=	Integrated Server
LOGP	=	Log Permission object
XPN	=	XPNET Interface object


IS Process
Log Messages
EVT
Objects
Log Messages
Network-level permission requests
(Log the event? Response is Yes or No)
XPN
LOGP
Log Permission Process
(LPP)
ESEMT
Read at
initialization
ECT
Network-level
permission requests
ACI Event Log Utilities
EMS
Collector
Event
Log
EMS
Distributor
EMS
Distributor
EMS
Distributor

Network Level Suppression

The Log Permission process reduces the number of messages logged in a network by determining whether the same log message should be logged from a number of different processes. If a number of processes generate the same event, the Log Permission process will only allow one of the processes to log the event between threshold intervals. This effectively prevents a burst of identical messages from being emitted by a group of processes at the same time.

The Log Permission object performs the same logic as the Event object in determining whether an event is to be suppressed or logged. Both objects utilize the Event Suppression File extended memory segment table built from the Event Suppression File (ESF). Both objects check whether the remaining number of messages to skip is 0 and whether the current time falls within the time interval for which the message is to be suppressed. If the remaining number of messages to skip is not 0 or the current time falls within the suppression time interval, the message is not logged. The Log Permission object loads information from the Event Suppression File extended memory table into an Event Control Table (ECT) during initialization processing. The ECT is then used during network-level event suppression arbitration.

Each Log Permission process handles event suppression detection for a particular range of subsystem IDs. This allows you to spread the burden of network-level event suppression detection across multiple Log Permission processes. Because each Log Permission process can handle event suppression requests from any IS process in the network, event suppression by the Log Permission process is effectively done at the network level rather than at the process level when performed by the Event object.

RESET ECT Command

The RESET ECT command allows you to reset values maintained in the Event Control Table (ECT). This command resets the next log timestamp and the current skip count such that the next occurrence of an event will be logged and not suppressed. This allows you to check whether corrective action you have taken has truly corrected an error condition and removes the possibility that the event is still occurring but is being suppressed. The first occurrence of an event after the ECT has been reset is always logged.

The RESET ECT command is entered by a network operator from the Server Control (SCS) screen. For detailed information on the SCS screen and its fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

The syntax for this command is RESET ECT, which must be entered in the FUNCTION NAME field on SCS screen 1.

The RESET ECT command is sent to the Log Permission object in the Command Message member function of the Process Control to Object lexicon.

Configuration

This subsection lists the configuration requirements for the Log Permission object. For detailed instructions on configuring the kernel environment and event suppression, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

Kernel Configuration

The Log Permission object must be configured to every Log Permission process running in the ITS environment.

A record must be added to the Object File (OBJ) for the Log Permission object. Once this OBJ record has been established, records associated with the object must be added to the following files:

- Process/Object Relationship File (PORF)
- EMT/Object Relationship File (EORF)

Event Suppression Configuration

All events that are to be suppressed must be defined in the Event Suppression File (ESF). If an event is not defined in the ESF, it will not be suppressed at either the process or network level.

The Log Permission Process Partition File (LPF) maps ranges of subsystem identifiers (SSIDs) to specific Log Permission processes responsible for arbitration when determining whether an event is to be logged.

LCONF Assigns and Params

The Log Permission object uses the ESEMT assign from the Logical Network Configuration File (LCONF) assign in its processing. For a description of this assign and for instructions on adding it to the LCONF, refer to the ***BASE24 Logical Network Configuration File Manual***.

Member Functions

This subsection documents the member functions sent and received by the Log Permission object.

Member Functions Sent

The table below shows the member functions sent by the Log Permission object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Object to ANS	Register
Object to Event	Log
Object to Space	Memory Free
	Memory Get
Object to Trace	Add Trace
Object to Transport Layer Object	Message Out

Member Functions Received

The table below shows the member functions sent by the Log Permission object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
ANS to Object	Initialize
Event to Log Permission	Event Log

Lexicon Name	Member Function
Process Control to Object	Command Message
	EMT Warmboot
	LCONF Warmboot
Transport Layer Object to Object	Message Failed
	Message In
Trace to Object	Start Trace
	Stop Trace

Section 6

Process Control Object

The Process Control object interprets and dispatches network operator-initiated and process-initiated commands to the appropriate objects. This section describes the Process Control object and includes the following topics:

- An overview of how the Process Control object fits into message processing
- A detailed discussion of how the Process Control object uses Integrated Server Control Files (ISCFs)
- A detailed discussion of the normal processing performed by the Process Control object
- A discussion of the configuration requirements for the Process Control object
- A list of member functions supported by the Process Control object

Overview

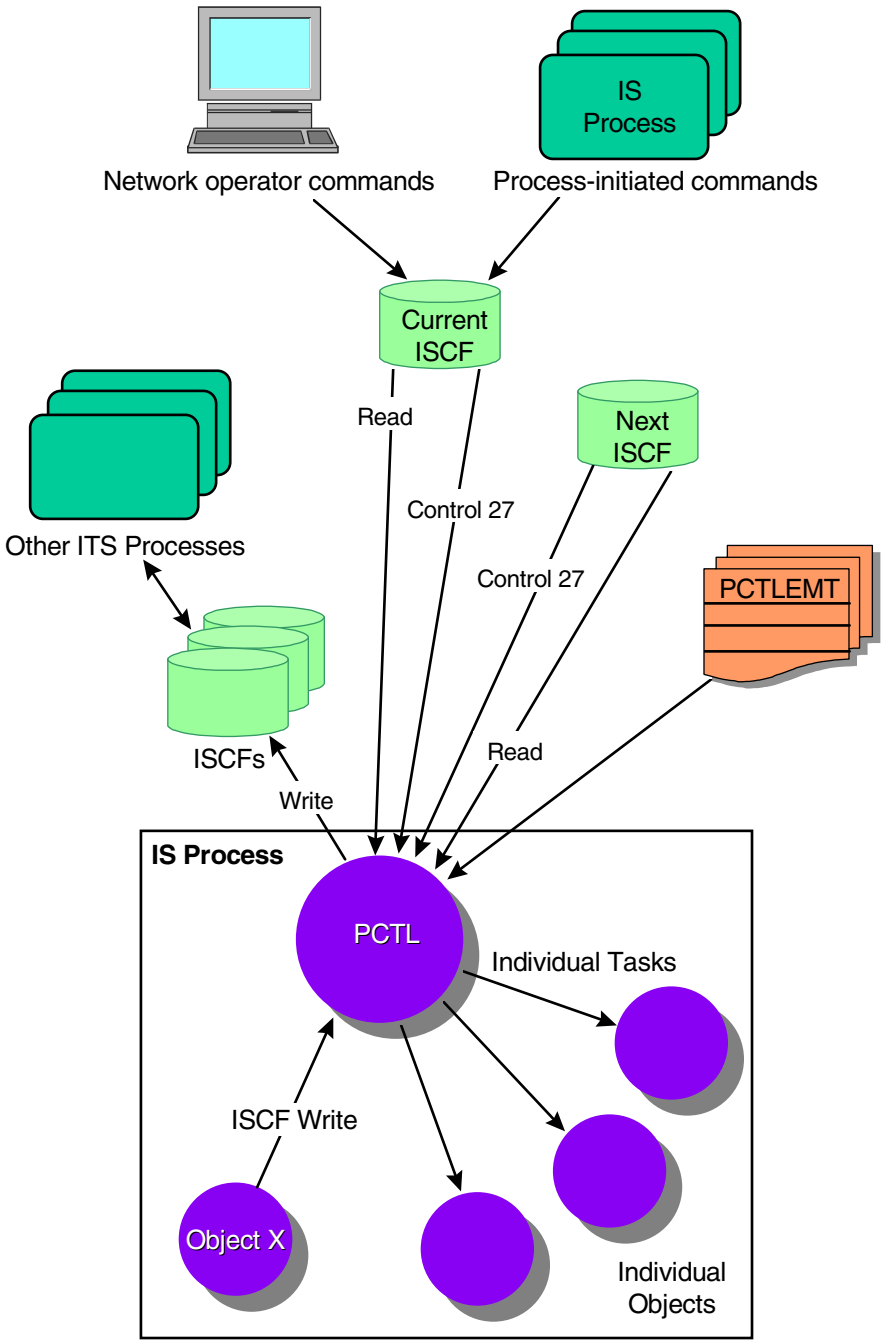
The Process Control object receives messages from an Integrated Server Control File (ISCF) and forwards them to the appropriate object. The ISCF is the mechanism by which commands entered by network operators are delivered to the objects within object-oriented ITS processes and by which commands or responses can be sent from one object-oriented ITS process to other ITS processes (including both object-oriented processes and procedural processes that can receive process control commands or responses).

The Process Control object must be bound in to every object-oriented process running in the ITS environment.

The functions of the Process Control object are depicted in the illustration on the following page. A key to the abbreviations is shown below.

Key

ISCF	=	Integrated Server Control File
IS	=	Integrated Server
PCTL	=	Process Control object
PCTLEMT	=	Process control extended memory segment


Network operator commands
IS Process
Process-initiated commands
Current ISCF
Read
Next ISCF
Control 27
Other ITS Processes
Control 27
PCTLEMT
ISCFs
Read
Write
IS Process
PCTL
Individual Tasks
ISCF Write
Object X
Individual Objects

ISCF Processing

The Integrated Server Control File (ISCF) is used as a means of communicating operator-initiated and object-initiated process control commands or responses to ITS objects as well as to procedural processes that can receive process control commands. All operator-initiated and object-initiated commands are written to the current ISCF for a server class. The Process Control object posts a read and a Guardian control 27 call on both its current and next control file to read incoming messages, and to detect any changes in the end-of-file, respectively. Any time the end-of-file changes, the Process Control object reads the ISCF, interprets the command, and notifies the appropriate object of the command to be executed. When the current ISCF fills up, the object receives notification from its control 27 call posted against the next ISCF. In this case, processing is rolled over to the next ISCF. ISCFs have a control number from 0 to 9. When an ISCF becomes full it wraps to the next highest number, with 9 wrapping back to 0. There are always two open ISCFs, one for current processing and another for when the current file becomes full.

One ISCF is configured for each server class in the ITS environment. Only processes that use the same types of messages and perform the same types of processing should be configured to the same server class. ISCFs are configured to server classes using the Server Class Type File (SCT). If a command written to the ISCF is directed to a particular process within a server class, the record will specify the process pair directory (PPD) name of the process. Otherwise, the PPD name is set to a value of ALL. If a command is intended for a particular object, the ISCF record will also specify the object ID.

Rollover Processing

Rollover processing is performed when a File I/O Completion member function is received on the posted control 27 call for the next ISCF rather than the current ISCF. During rollover processing, the Process Control object performs the following:

1. Closes the current ISCF.
2. Starts using the next ISCF. The next ISCF becomes the current ISCF.
3. Opens the new next ISCF.
4. Assigns a file number to the new next ISCF. The new next ISCF becomes the current next ISCF.
5. Creates a new next ISCF.

ISCF Write Processing

The Process Control object performs the following when it receives an ISCF Write member function from another object in the ITS process. The ISCF Write member function allows objects to send information to objects in other processes or to procedural processes (for example, the Enscribe Refresh process).

1. Initializes and allocates the process control extended memory table.
2. Formats the ISCF record using information from the ISCF Write member function.
3. Determines the ISCFs to which the member function is written, as follows:
    a. If a symbolic process name is provided in the member function, the object performs the following:
        1) Reads the Server Class Type/Process Relationship File (SPRF) using the alternate key to retrieve the server class type.
        2) Reads the Server Class Type File (SCT) using the primary key to retrieve the control file assign name and the control file sequence number.
        3) Reads the Process File (PRO) using the primary key to retrieve the process pair directory (PPD) name for the process.
        4) Using the control file assign name from the SCT, the object reads the Logical Network Configuration File (LCONF) to retrieve the ISCF file and template names.
        5) Opens the specified ISCF, writes the command to the ISCF, and closes the ISCF.
    b. If an object ID is provided in the member function, the object performs the following:
        1) Reads the Object File (OBJ) using the object ID alternate key to retrieve the object name.
        2) Reads the Process/Object Relationship File (PORF) to obtain the process names.
        3) If a process type was included with the member function, the object eliminates any processes from the previous step that are not of the designated type.
        4) For each valid process name, the object reads the SPRF using the alternate key to retrieve the server class type.

5) For each valid server class type, the object reads the SCT using the primary key to retrieve the control file assign name and the control file sequence number.

6) Using the control file assign names from the SCT, the object reads the LCONF to retrieve the ISCF file and template names.

7) For each ISCF, the object opens the ISCF, writes the command to the ISCF, and closes the ISCF.

ISCF Write Failures

When a write operation fails on an ISCF due to a file full condition, processing similar to rollover processing previously discussed is attempted. The only difference is that the ISCF found in a SCT record is used rather than the ISCF found in the object's frame.

Process Control Extended Memory Table Processing

The process control extended memory table is simply used as work area to ensure that a command is written to the proper ISCFs. A separate ISCF is maintained for each server class running in the ITS environment.

Normal Processing

When a network operator initiates a command or when an object needs to inform objects in other processes of an event for which actions need to be taken, the Process Control object receives a File I/O Completion member function. When the object detects a change in its Integrated Server Control File (ISCF) end-of-file, it knows a new command message has been received.

When a command is written to the ISCF, the Process Control object reads the file, interprets the command, and forwards the appropriate lexicon member functions to the appropriate objects to execute the command. The following member functions can be forwarded to other objects bound into the same process or belonging to the same server class, depending on the nature of the request.

- Assign Warmboot
- Command
- EMT Warmboot
- File Close/Open
- LCONF Warmboot
- Status

If the command is for object tracing, any of the following Trace member functions can be forwarded to other objects bound into the same process, or belonging to the same server class:

- Trace List
- Trace Off
- Trace On
- Trace On Secure Internal Transaction Data (ITD)

The processing performed for each of the above member functions is described in section 1.

Configuration

This subsection lists the configuration requirements for the Process Control object.

Kernel Configuration

The Process Control object must be configured to every ITS process.

A record must be added to the Object File (OBJ) for the Process Control object. Once this OBJ record has been established, records associated with the object must be added to the Process/Object Relationship File (PORF). For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

LCONF Assigns and Params

The Process Control object uses the following Logical Network Configuration File (LCONF) assigns or params in its processing. For complete descriptions of these assigns and params and instructions for adding them to the LCONF, refer to the ***BASE24 Logical Network Configuration File Manual***.

Assigns	**Params**
ISCF-*server class*	PCTLEMT-SEGSIZE
OBJ	
PCTLEMT-SWAPVOL	
PORF	
PRO	
SCT	
SPRF	

The ISCF-*server class* assign can be reread using the ASSIGN WARMBOOT command.

Member Functions

This subsection documents the member functions sent and received by the Process Control object. Each of these member functions is described in section 1.

Member Functions Sent

The table below shows the member functions sent by the Process Control object, arranged according to the lexicon of which they are a part. It also lists the Data Definition Language (DDL) names of the request and response structures available for online viewing. A description of each member function is included in section 1.

Lexicon Name	Member Function	DDL Name
All lexicons	Version	LEX-HDR
Object to Application Network Services	Register	Not available for online viewing.
Object to Event	Log	Not available for online viewing.
Object to Space	Memory Get	Not available for online viewing.
Object to Trace	Add Trace	Not available for online viewing.
Process Control to Application Network Services	Debug On	Not available for online viewing.

Lexicon Name	Member Function	DDL Name
Process Control to Object	Assign Warmboot	PCTL-ASGN-WB-RQST PCTL-ASGN-WB-RESP
	Command	PCTL-CMD-RQST PCTL-CMD-RESP
	EMT Warmboot	PCTL-EMT-WB-RQST PCTL-EMT-WB-RESP
	File Close/Open	PCTL-FILE-CLSOPN-RQST PCTL-FILE-CLSOPN-RESP
	LCONF Warmboot	PCTL-LCONF-WB-RQST PCTL-LCONF-WB-RESP
	Param Warmboot	PCTL-PARAM-WB-RQST PCTL-PARAM-WB-RESP
	Status	PCTL-STAT-RQST PCTL-STAT-RESP
Process Control to Trace	Trace List	PCTL-TRC-LIST-RQST PCTL-TRC-LIST-RESP
	Trace Off	PCTL-TRC-OFF-RQST PCTL-TRC-OFF-RESP
	Trace On	PCTL-TRC-ON-RQST PCTL-TRC-ON-RESP
	Trace On Secure ITD	PCTL-TRC-ON-SEC-ITD-RQST PCTL-TRC-ON-SEC-ITD-RESP

Member Functions Received

The table below shows the member functions received by the Process Control object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services to Object	Initialize
	Stop
Object to Process Control	ISCF Write
Process Control to Object*	LCONF Warmboot
Trace to Object	Start Trace
	Stop Trace
Transport Layer Object to Object	File I/O Completion

*This lexicon is sent from the Process Control object to itself.

ACI Worldwide Inc.

Section 7

Trace Object

The Trace object provides trace facilities for the integrated transaction services (ITS) processes. This section describes the Trace object and includes the following topics:

- An overview of how the Trace object fits into message processing
- A discussion of the various trace levels supported by the Trace object
- A detailed discussion of how the Trace object interacts with objects undergoing a trace using TRACE commands
- A discussion of the configuration requirements for the Trace object
- A list of member functions supported by the Trace object

Overview

The Trace object is part of the general trace facility available to all objects in the ITS environment. The trace facility is called by a command from the Process Control object. The Trace object writes the information being traced to a trace file or directly to a terminal.

TRACE commands are entered by network operators and delivered by the Integrated Transaction Services Control Server process to an Integrated Server Control File (ISCF). The Process Control object reads the ISCF and forwards the appropriate trace member function to the Trace object. The Trace object sends start and stop trace member functions to other objects in the process and receives Add Trace member functions from other objects when an object has data to be included in the trace. When the Trace object receives the Trace List member function, it generates an EMS event message listing the current status of the trace in progress.

The Trace object uses trace facilities to create a trace file or to display trace data on a terminal screen. Data received in Add Trace member functions is flushed to the trace file or terminal.

Trace files are named in the format of TRC*xxxxn*, where *xxxx* is the PPD name of the process being traced, and *n* is a sequential number in the range of 0–9. All trace files are placed on the volume and subvolume specified in the TRACE FILE field on Trace Control (TRC) screen 2.

If trace information is to be displayed on a terminal screen, the trace information is sent to TERM *terminal name*, where *terminal name* is the actual name of the terminal where the trace data is to be displayed. The trace terminal file name is also specified in the TRACE FILE field on TRC screen 2.

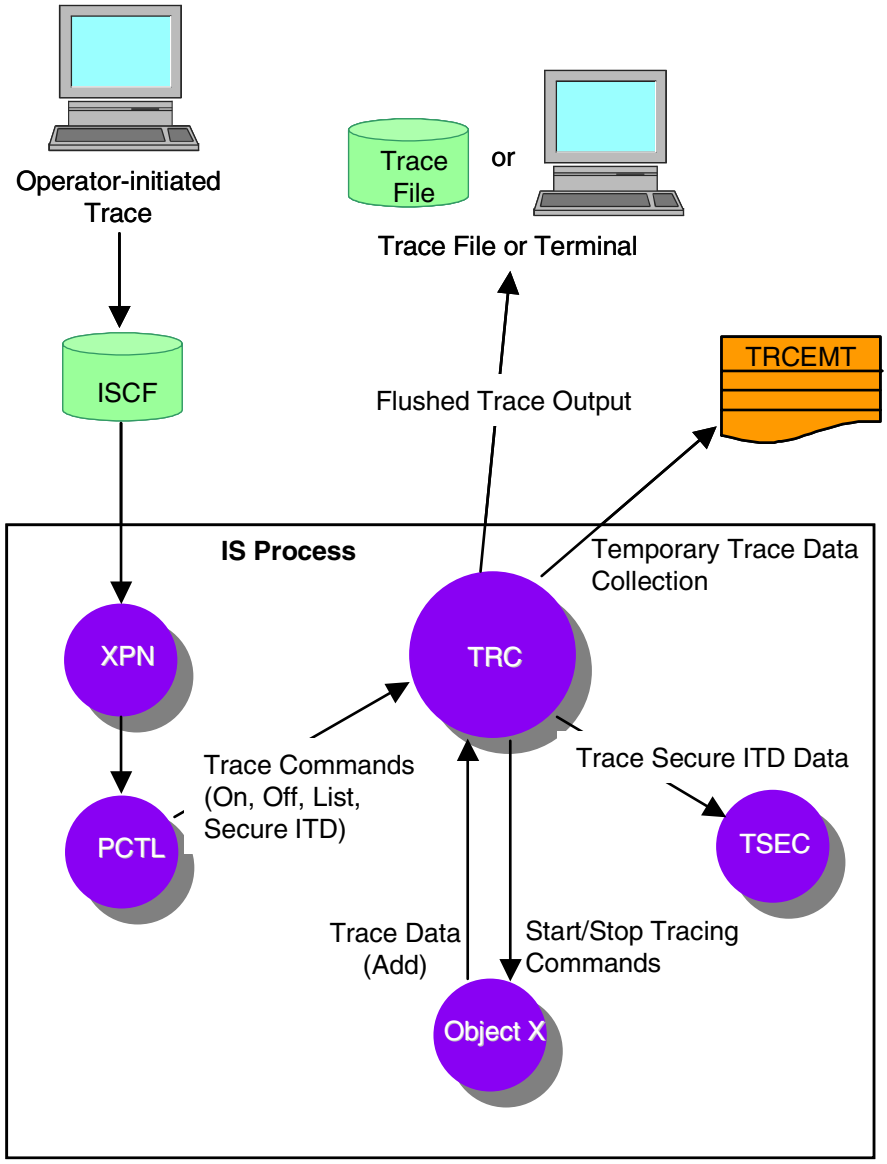
The Trace object uses the trace extended memory table when collecting trace data. The TRCEMT-SWAPVOL param specifies the location of a swap file used for this table. When the trace is stopped, the trace information temporarily stored in the trace extended memory table is flushed to the specified trace file or terminal.

When tracing to a file, the trace must be stopped before the trace file can be viewed or displayed. When tracing to a terminal, only procedure level, request or response lexicon headers, and external messages can be displayed.

Starting a trace only affects one process at a time. Starting a trace on a particular process does not cause any other processes in the same server class to begin tracing.

You can think of the Trace object as the internal auditor for an object-oriented ITS process. Trace data is used by trained personnel to pinpoint processing problems or to verify that processing is proceeding as planned. The format and content of trace output data goes beyond the scope of this manual. The Trace object must be bound into every object-oriented process running in the ITS environment.

The functions of the Trace object are depicted in the illustration on the following page. A key to the abbreviations follows the illustration.



Key

ISCF	=	Integrated Server Control File
IS	=	Integrated Server
PCTL	=	Process Control object
TRC	=	Trace object
TRCEMT	=	Trace extended memory table
TSEC	=	Transaction Security object
XPN	=	XPNET Interface object

Trace Levels

All of the possible trace levels for an ITS object are shown in the following table. Trace levels 1 and 2 are typically supported by all objects. Objects may support additional trace levels as required and configured for the object in the Object File (OBJ). Detailed descriptions for each of these trace levels are provided with the OBJ screen documented in the ***BASE24 ITS Kernel Files Maintenance Manual***. The trace levels supported for the default OBJ records provided by ACI are also documented in this manual.

Trace Level	Description
1	Trace all procedural calls.
2	Trace all object interactions.
3	Trace all disk file input and output.
4	Trace all interprocess input and output.
5	Trace Internal Transaction Data (ITD).
6	Trace external Message In.
7	Trace external Message Out.
8–16	Reserved for use of globally supported trace levels.
17–32	Reserved for object-specific trace levels.
33–64	Reserved for future use.
99	Trace all trace levels defined.

Trace Processing

Trace processing is controlled by member functions sent to and from the Trace object. Each of these member functions and the processing performed on behalf of the functions is summarized below. Descriptions for each of these member functions are provided in section 1.

- The Trace object receives the Trace On member function from the Process Control object when an operator initiates a trace. The Trace object then sends a Start Trace member function to each object to be traced.
- Once a trace has been started, the Trace object receives the Add Trace member function from a traced object when the object has data to be added to the trace. This information is collected in the trace extended memory table and is periodically flushed to the trace file or terminal.
- The Trace object receives the Trace List member function from the Process Control object when a network operator requests the current status of a trace in progress. The response data to the Trace List member function includes the percentage full of the trace extended memory table completed thus far, whether file wrapping is allowed, and whether file wrapping has occurred. Trace list response information is logged in an Event Management Service (EMS) event message.
- The Trace object receives the Trace On Secure ITD member function from the Process Control object when an operator requests a trace of secure Internal Transaction Data (ITD) from the Trace Control screen. The Trace object then sends a Start Secure ITD member function to the appropriate object (for example, the Transaction Security object) to initiate tracing of an object's secure data, such as cardholder PIN validation information from the internal transaction data (ITD) exchanged between objects.
- The Trace object receives the Trace Off member function from the Process Control object when a trace is stopped by an operator using the Trace Control screen. The Trace object then sends the Stop Trace member function to all objects currently being traced. All levels for all objects in the affected process are stopped when tracing is stopped.

Configuration

This subsection lists the configuration requirements for the Trace object.

Kernel Configuration

The Trace object must be configured to every object-oriented process running in the ITS environment.

A record must be added to the Object File (OBJ) for the Trace object. Once this OBJ record has been established, records associated with the object must be added to the Process/Object Relationship File (PORF). For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

LCONF Assigns and Params

The Trace object uses the TRCEMT-SWAPVOL assign from the Logical Network Configuration File (LCONF) in its processing. For a description of this assign and for instructions on adding it to the LCONF, refer to the ***BASE24 Logical Network Configuration File Manual***.

Member Functions

This subsection documents the member functions sent and received by the Trace object.

Member Functions Sent

The table below shows the member functions sent by the Trace object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Object to Event	Log
Object to Application Network Services Object	Register
Trace to Application Network Services Object	Request/Response Start
Trace to Object	Start Trace
	Start Secure ITD
	Stop Trace
Object to Space	Memory Free
	Memory Get

Member Functions Received

The table below shows the member functions received by the Trace object, arranged according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
ANS to Object	Initialize
	Stop ANS
Object to Trace	Add Trace
Process Control to Object	LCONF Warmboot
Process Control to Trace	Trace List
	Trace Off
	Trace On
	Trace On Secure ITD

ACI Worldwide Inc.

Section 8

XPNET Interface Object

The XPNET Interface object provides an interface between the XPNET transport layer and an integrated transaction services (ITS) process running under control of the XPNET process. This section describes the XPNET Interface object and includes the following topics:

- An overview of how the XPNET Interface object fits into message processing
- A detailed discussion of how the XPNET Interface object supports communication between the XPNET transport layer and the ITS process during normal processing
- A discussion of network overload management processing performed by the XPNET Interface object
- A discussion of the memory requirements for message buffers used by the XPNET Interface object
- A discussion of the network header services provided by the XPNET Interface object
- A discussion of the configuration requirements for the XPNET Interface object
- A list of member functions supported by the XPNET Interface object

Overview

The XPNET Interface object is responsible for sending and receiving messages through the $RECEIVE file when a process runs under control of the XPNET process. The object serves as an interface between the ITS application process and the XPNET transport layer. It is responsible for the following:

- Sending outgoing messages to the XPNET network
- Replying to all XPNET network and non-network messages received
- Retrieving the source class or object ID for all incoming application messages
- Detecting network overload on incoming messages for a process or queue
- Providing XPNET network header services to other objects within the process

If a symbolic name is supplied in a message, the message either originated from the XPNET process or is destined for the XPNET process, and is considered to be a network data message. Otherwise, the message is a non-network data message or a non-network system message. Currently, the XPNET Interface object does not expect to receive any network system messages. If a network system message is received, an exception is raised.

Non-Network Message Processing

A non-network data message is any message received from another process that was not delivered through the XPNET network layer. For example, a message sent directly from an Integrated Authorization Server process to an Interface Master process would fall under this category. A non-network system message is a message received directly from the operating system without going through the XPNET network layer. Time signal and process time signal messages would fall under this category. Non-network messages are dispatched to the proper object based on the source of the message or the message content.

Non-network data and system messages are sent directly to the object that requires them.

Network Data Message Processing

For all network data messages received, the XPNET Interface object retrieves the source class of the message using the symbolic source in the message header and the source symbolic map extended memory table. When a network stop message is received, the XPNET Interface object sends a Stop Message member function to the Application Network Services object.

For all network data messages received, the XPNET Interface object provides network overload detection. Using percentages provided by the XPNET process in the network header of each network data message received and an LCONF param, the XPNET Interface object determines whether network overload exists for the process or queue. If so, the object sets a field in the header that other objects in the process can use to take the appropriate network overload action, such as discarding the message, returning it to its source, or saving the message to a file.

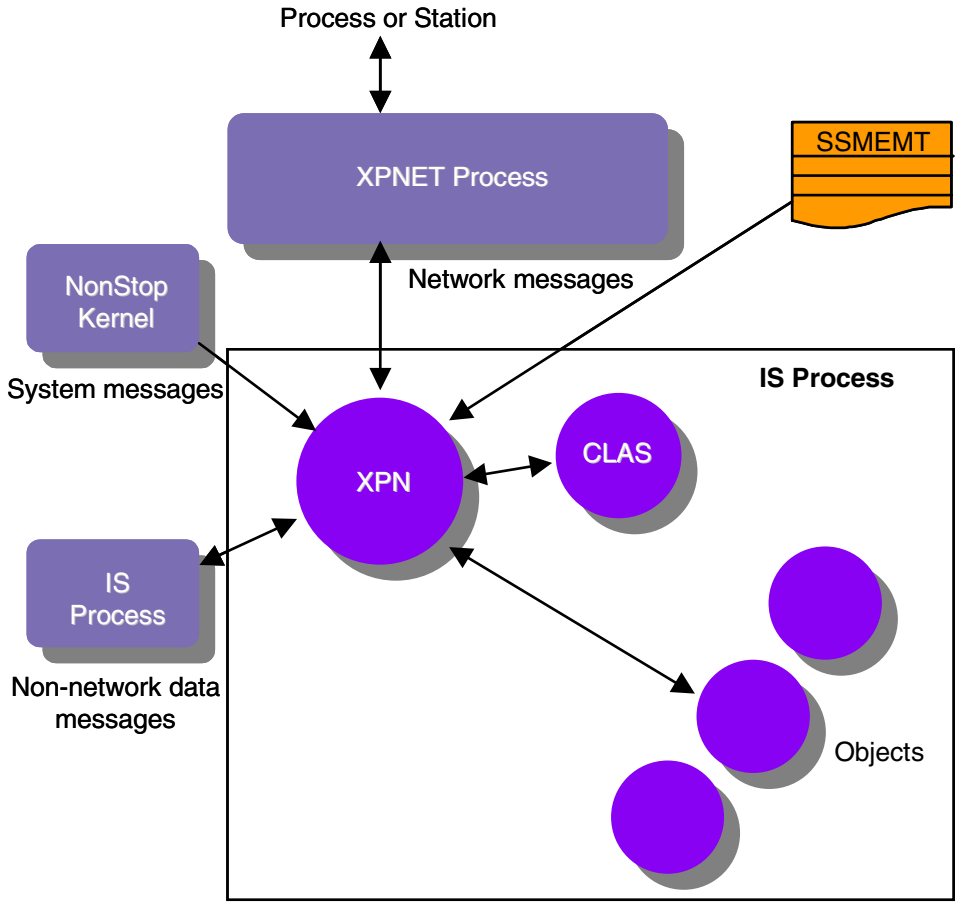
The XPNET Interface object also provides network header services, allowing objects within the process to request specific information from the header of an incoming network data message or set specific information in the header of an outgoing network data message.

Incoming Message Replies

The XPNET Interface object is responsible for replying to all messages it receives. A single transaction may produce no messages or one or more outbound messages. Replies are always sent immediately to the same location from which the associated request originated.

Interactions

The interactions of the XPNET Interface object are depicted in the illustration on the following page. A key to the abbreviations follows the illustration.



Key

CLAS	=	Class object
IS	=	Integrated Server process
SSMEMT	=	Source symbolic map extended memory table
XPN	=	XPNET Interface object

Source Class Retrieval

The following paragraphs discuss the source class retrieval processing performed for network and non-network data messages. Source class retrieval does not apply to non-network system messages.

Network Data Messages

For all network messages received from the XPNET process, the XPNET Interface object retrieves the message symbolic source from the message header.

Using the symbolic source name, the object reads the source symbolic map extended memory table to retrieve the source class. This table is established from information in the Network Environment File (NEF) and is used to determine the source class for an incoming message. The source class is derived from the SOURCECLASS attribute configured for a process or station in the XPNET system.

After the source class is retrieved from the source symbolic map extended memory table, the XPNET object calls the Class object with the Object ID Retrieve member function to retrieve the ID of the object to handle messages from that source. The message is then routed by the XPNET Interface object to that message handler object.

Non-Network Data Messages

Non-network data messages received from other object-oriented processes are enveloped within a Message member function. The Message function is a member of the Application to Object lexicon.

This function is sent from an object-oriented application process to an object in an object-oriented process and is received by the XPNET Interface object. A writeread interface rather than the XPNET process is used to deliver this message. The Message member function contains either the object ID of the destination object to receive the message or a source class. If a source class is specified in the Message member function, the XPNET Interface object obtains the destination object for the message by calling the Class object with the Object ID Retrieve member function as described above. If the source class is not specified in the member function, the destination object is specified by the destination object ID

provided in the member function. In either case, the XPNET Interface object delivers the message to the destination object by enveloping the member function within a Message In member function and sending it to the destination object.

Network Overload Detection

For all network data messages received, the XPNET process provides the current network overload percentage in the Network Overload Management Percent field of the network header for destination processes. The XPNET Interface object compares the value of this field to the value of the NOM-PERCENT-THRESHOLD param from the LCONF.

If this param is set to a value of 0, network overload management processing is not performed and the XPNET Interface object sets the Network Overload Management Percent field in the header to a value of 0. If this param is set to a nonzero value and the current network overload management percentage in the message header is equal to or greater than the value of this param, the XPNET Interface object sets this field to a value of 101. A value of 101 indicates that overload was detected. The appropriate message handler object (that is, an Endpoint Class Handler object, terminal object, or interface object) can then take the appropriate network overload management actions for the process.

The XPNET process updates the Network Overload Management Percent field in the message header with the overload management information before the message is delivered to the ITS process. Based on the settings of the QMT and QMI attributes configured for the process in the XPNET system, the XPNET process calculates two percentages: the percentage for messages queued in relation to the queue management threshold for the queue and the percentage of memory used in relation to the queue management index for the queue. The lower of these two percentages is placed in the Network Overload Management Percent field.

A value of 0 for the above attributes, effectively disables network overload management for a process, regardless of the value of the NOM-PERCENT-THRESHOLD param.

For more information on how the XPNET process uses the Network Overload Management Percent field, refer to the ***NET24-XPNET Application Programming Guide***.

Network Data Message Header Services

The header of all network data messages contains information that can be vital to transaction processing, such as the following:

- The symbolic destination for a message
- Whether network overload exists
- Whether a message should be force queued when network overload exists
- Whether an acknowledgment is requested for the message

The XPNET Interface object provides the means to retrieve and set specific network data header information through the use of various member functions in the Object to XPNET Interface lexicon. Each of these member functions either retrieves specific data from the network header or sets specific data in the network header. Any object in the ITS process can call the XPNET Interface object using these member functions. Each member function provided in this lexicon is described in section 1.

After an incoming network data message has been read from the input message buffer, any other objects in the process can retrieve specific information from the network header of the message. This is accomplished by sending the appropriate Object to XPNET Interface lexicon member functions to retrieve data from the message header.

After an outgoing network data message has been processed and is placed in the output message buffer, any other objects in the process can also set specific information in the network header. This is accomplished by sending the appropriate Object to XPNET Interface lexicon member functions to set data in the message header.

Library Usage

The XPNET Interface object uses the XPNET network library routines typically located in the XNLIB subvolume on your system volume. The object uses the XNLIB file in this subvolume for systems running under XPNET 3.0.

An XPNET network library routine file must be specified in the LIBRARY attribute for all object-oriented ITS processes in the XPNET system. For more information on the configuration of the LIBRARY attribute on behalf of an ITS process, refer to the ***BASE24 ITS Kernel Configuration Manual***. For more information on the XPNET network library routines, refer to the appropriate informal ACI documentation.

Configuration

This subsection lists the configuration requirements for the XPNET Interface object.

Kernel Configuration

The XPNET Interface object must be configured in every ITS process running under control of the XPNET process.

A record must be added to the Object File (OBJ) for the XPNET Interface object. Once this OBJ record has been established, records associated with the object must be added to the following files:

- Process/Object Relationship File (PORF)
- EMT/Object Relationship File (EORF)

For detailed instructions on configuring the kernel environment, refer to the ***BASE24 ITS Kernel Configuration Manual***. For descriptions of the kernel file maintenance configuration screens and fields, refer to the ***BASE24 ITS Kernel Files Maintenance Manual***.

LCONF Assigns and Params

The XPNET Interface object uses the SSMEMT assign and the QED and NOM-PERCENT-THRESHOLD params from the Logical Network Configuration File (LCONF) in its processing. For descriptions of these assigns and params, and for instructions on adding them to the LCONF, refer to the ***BASE24 Logical Network Configuration File Manual***.

The NOM-PERCENT-THRESHOLD and QED params can be reread using the PARAM WARMBOOT command.

Member Functions

This subsection documents the member functions sent and received by the XPNET Interface object.

Member Functions Sent

The table below shows the member functions sent by the XPNET Interface object, according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Object to Application Network Services	Register
Object to Class	Object ID Retrieve
Object to Event	Log
Object to Space	Memory Get
Object to Trace	Add Trace
Transport Layer Object to Application Network Services	Stop Message
	System Message
Transport Layer Object to Object	File I/O Completion
	Message Failed
	Message In
	System Time Signal
Transport Layer Object to Transport Layer Object	Message to Object

Member Functions Received

The table below shows the member functions received by the XPNET Interface object, according to the lexicon of which they are a part. A description of each member function is included in section 1.

Lexicon Name	Member Function
All lexicons	Version
Application Network Services to Object	Stop
Application to Object	Message
Object to Transport Layer Object	Message Out
Object to XPNET	Message Header Control Flag On
	Message Header Get Destination Queue State
	Message Header Get Message Disposition
	Message Header Get Message Failure
	Message Header Get Message Priority
	Message Header Get Message Process Words
	Message Header Get Message Sequence Number
	Message Header Get Message Timeout
	Message Header Get Message Timestamp
	Message Header Get Message Transmission Number

Continued on the following page

Lexicon Name	Member Function
Object to XPNET *continued*	Message Header Get Source Type
	Message Header Get Symbolic Destination
	Message Header Get Symbolic Source
	Message Header Logical Acknowledgment Requested
	Message Header Message Force Queue On
	Message Header Message Function Management Present
	Message Header Message Possible Duplicate
	Message Header Message Transparency On
	Message Header SAF Indicator On
	Message Header Set Control Flag
	Message Header Set Logical Acknowledgment
	Message Header Set Message Audit
	Message Header Set Message Disposition
	Message Header Set Message Failure
	Message Header Set Message Force Queue
	Message Header Set Message Priority
	Message Header Set Message Process Words

Continued on the following page

Lexicon Name	Member Function
Object to XPNET *continued*	Message Header Set Message Timeout
	Message Header Set Message Transparency
	Message Header Set NOM Override
	Message Header Set NOM Return
	Message Header Set SAF Indicator On
	Message Header Set Symbolic Destination
Process Control to Object	EMT Warmboot
	LCONF Warmboot
	Param Warmboot
Transport Layer Object to Object	System Time Signal
Transport Layer Object to Transport Layer Object	Message to Object
Trace to Object	Start Trace
	Stop Trace

Section 9

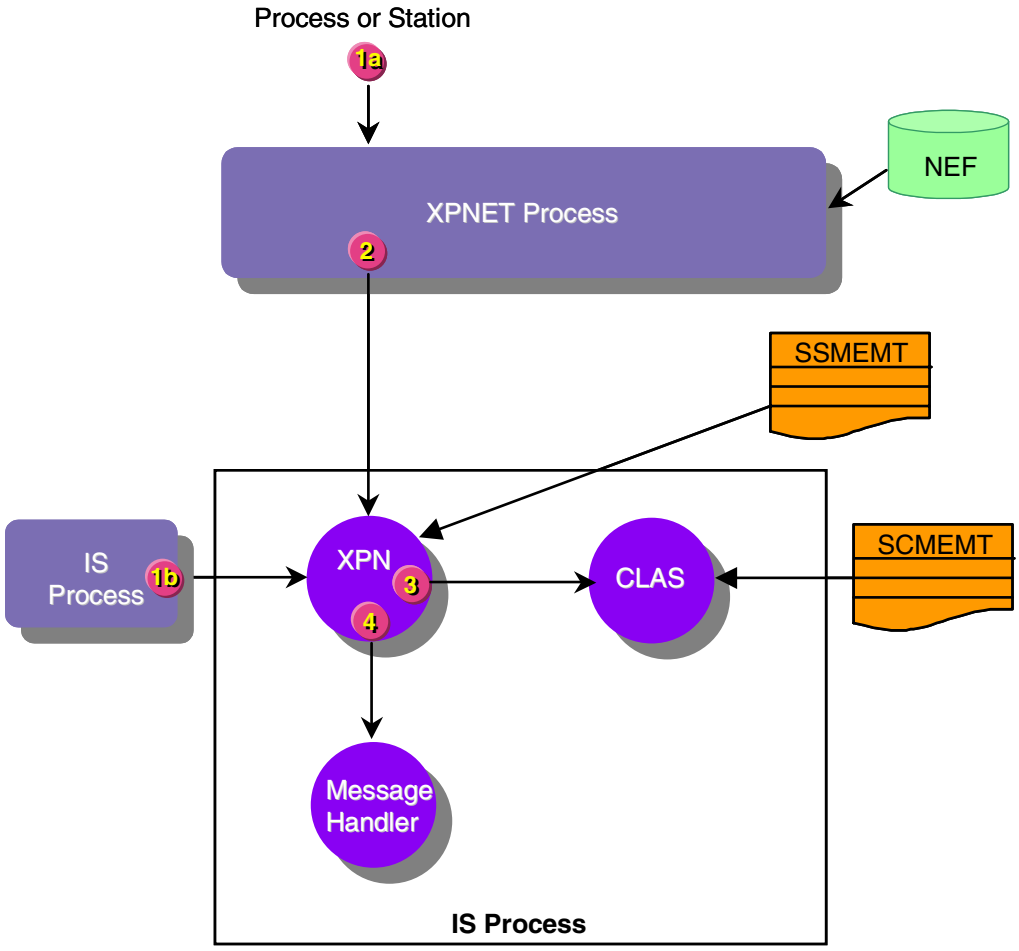
Processing Flows

This section provides you with various scenarios that highlight the processing performed by kernel objects. These examples are presented as an aid to understanding the flow of information through the integrated transaction services (ITS) environment. Transaction processing flows are provided for the following scenarios:

- Object ID retrieval processing performed by the XPNET Interface object. This processing is performed on every network or data message received from an external entity.
- Process control processing performed by the XPNET Interface object and the Process Control object.
- Starting and stopping trace. This processing involves the Process Control, XPNET Interface, and Trace objects.
- Logging of event messages. This processing involves the following objects: Event, Log Permission, and XPNET Interface objects.
- Application initialization. This flow demonstrates the general processing performed when an ITS process is initialized. Specific object initialization steps may also be performed, depending upon the type of object initializing.

Object ID Retrieval Processing Flow

When an ITS process receives a new message, the XPNET Interface object must determine the object to which the message should be sent. The XPNET Interface object determines the destination object through object ID retrieval. Object ID retrieval processing is illustrated below and described on the following page. A key to the abbreviations follows the illustration.



Key

CLAS	=	Class object
CSR	=	Customer Service Representative
IS	=	Integrated Server
NEF	=	Network Environment File
RBSI	=	Remote Banking Standard Interface
SCMEMT	=	Source Class Map File extended memory table
SSMEMT	=	Source symbolic map extended memory table
XPN	=	XPNET Interface object

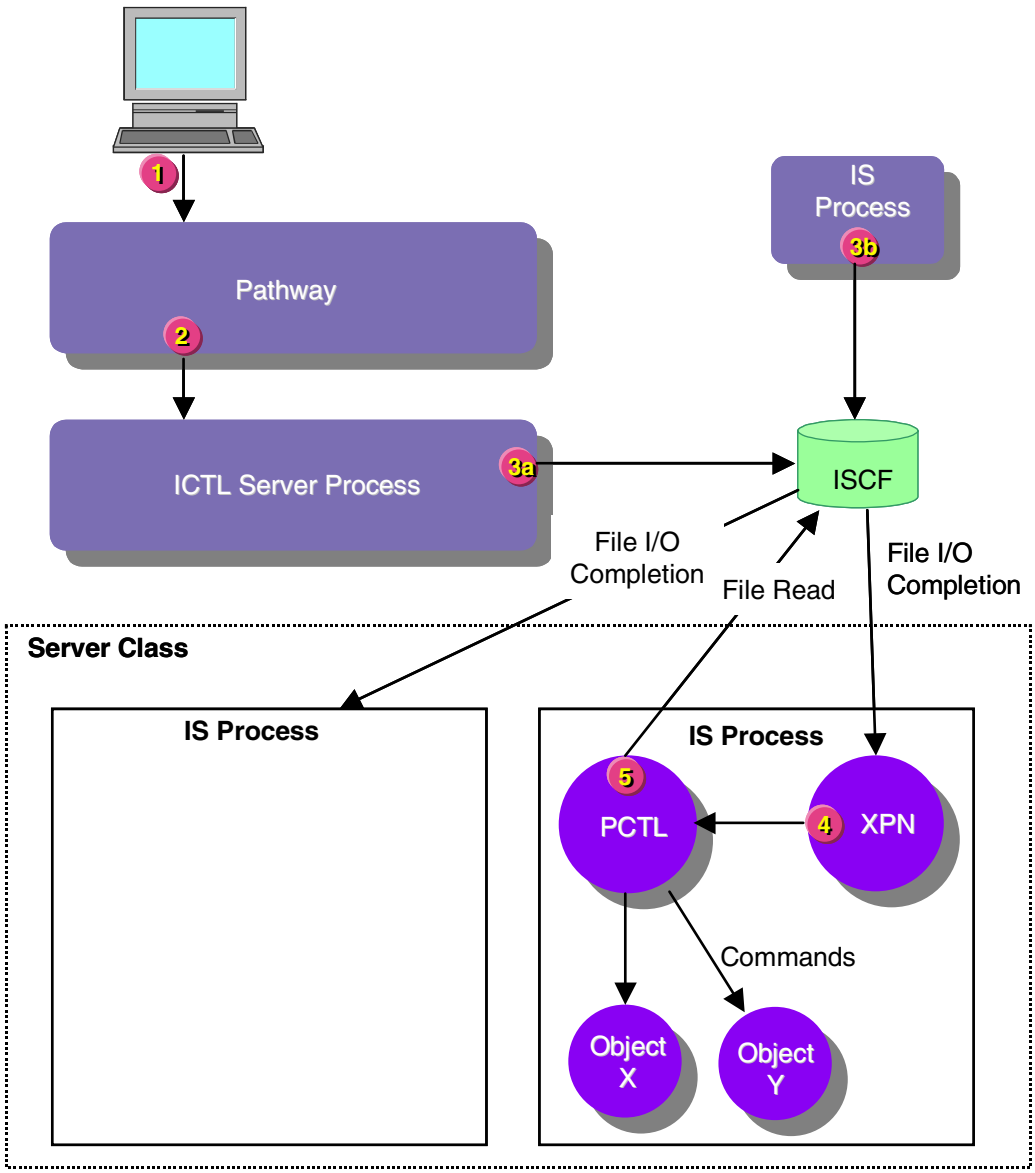
Processing involves the following steps:

1. A station, Integrated Server process, or procedural process running under the XPNET process sends a message destined for an Integrated Server process.
    a. A station, Integrated Server process, or procedural process sends the message to the XPNET process for delivery to the Integrated Server process. This message is considered to be a network data message.
    b. An object in an Integrated Server process sends a message directly to another Integrated Server process using the writeread procedure. In this case, the message does not pass through the XPNET process and is considered to be a non-network data message. This message is enveloped within the Message member function.
2. The XPNET process receives the network data message and determines the destination for the message, which in this case, is an Integrated Server process.
3. For data messages received from the XPNET process or from a writeread procedure, the XPNET Interface object in the Integrated Server process reads the incoming message from the $RECEIVE file. To determine the object to which to send the transaction, the XPNET Interface object performs the following:
    a. Determines the source class for the message or retrieves the destination object ID if it is included in the message. If the destination object ID is included in the message, proceed to step 4. Otherwise, continue with the following steps.
    b. Reads the source symbolic map extended memory table to obtain the source class for the message. If the source class is not found in this search, it must be provided in the message itself.
    c. Retrieves the ID of the message handler object for the message using the source class. The XPNET Interface object formats an Object ID Retrieve member function and calls the Class object with the source class obtained in the previous step. The Class object uses the source class to retrieve the destination object ID from the Source Class Map File extended memory table. The Class object returns the destination object ID to the XPNET Interface object in the Object ID Retrieve response.
4. Formats a Message In member function for the message and passes control to the destination object identified. The destination object is always a message handler object. Transaction processing proceeds as dictated by the message handler object.

Process Control Flow

When a network operator or process initiates a command, the command is written to the appropriate Integrated Server Control File (ISCF). There is a different ISCF for each server class in the ITS system. The XPNET Interface object in the appropriate Integrated Server process receives a completion due to a change in the end-of-file on its ISCF. The XPNET Interface object then passes the completion to the Process Control object that reads the ISCF, obtains the entered command, and passes the command on to each object in the local process that needs to execute it. Process control processing is illustrated on the following page. A key to the abbreviations follows the illustration. The processing steps depicted in the illustration are explained following the key.

Note: Process control commands can also be delivered to procedural processes using the ISCF for a procedural server class. This processing is not depicted in this flow.



Key

ICTL	=	Integrated Transaction Services Control Server process
ISCF	=	Integrated Server Control File
IS	=	Integrated Server process
PCTL	=	Process Control object
XPN	=	XPNET Interface object

Processing involves the following steps:

1. An operator initiates a process control operation at a Pathway terminal.

2. Pathway delivers the command to the Integrated Transaction Services Control Server process.
3. An operator-initiated or process-initiated command is written to the appropriate Integrated Server Control File (ISCF). The appropriate ISCF is determined by the configured server class of the component (process or object) to which the command is directed.
    a. For operator-initiated commands, the Integrated Transaction Services Control Server process writes the command to the appropriate ISCF.
    b. For process-initiated commands, the Integrated Server process writes the command to the appropriate ISCF.
4. An Integrated Server process within the designated server class receives a completion due to a change in the end-of-file on its ISCF. A file I/O completion is sent to every process in the server class.

    The XPNET Interface object receives the file completion and sends a File I/O Completion member function to the Process Control object.
5. The Process Control object receives the member function, reads the ISCF until it reaches the end-of-file, obtains the entered commands, and passes each command to the appropriate object where the specified command is executed.

Network Trace Flow

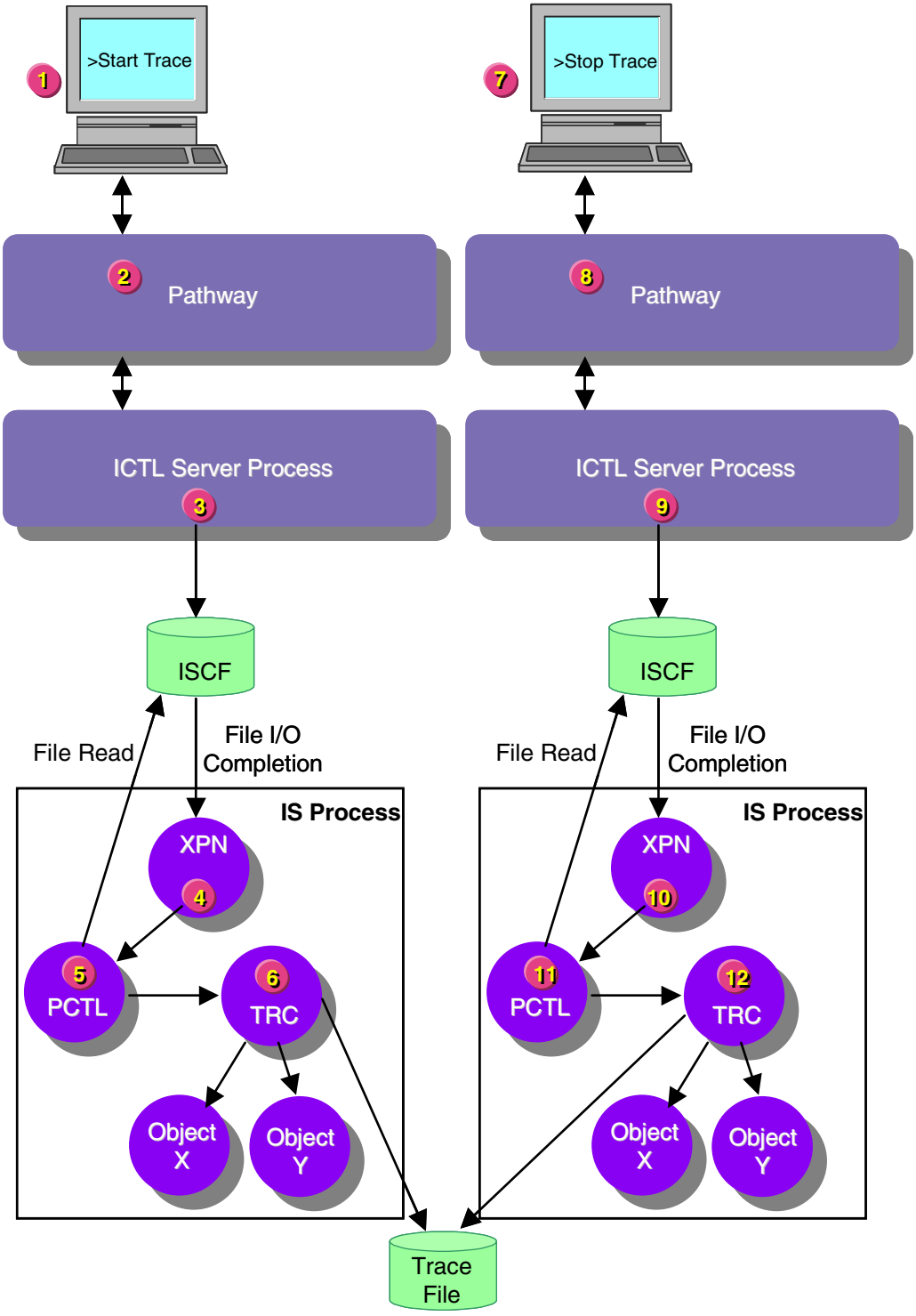
When an operator initiates a network trace on a particular process, several ITS components are involved in executing the trace. The Integrated Transaction Services Control Server process writes the request to the appropriate Integrated Server Control File (ISCF). The XPNET Interface object detects the change in the ISCF end-of-file and informs the Process Control object. The Process Control object reads the ISCF, obtains the TRACE command, and informs the Trace object of the request. The Trace object opens a trace file or terminal and informs the appropriate objects to begin tracing. During the trace, the traced objects send trace information to the Trace object, which adds the data to the trace file or sends the data to a terminal.

Note: This network trace flow example assumes that trace data is being written to a trace file rather than being displayed on a trace terminal.

The same ITS components are involved when a network operator stops a trace. Trace processing is illustrated and described on the following pages. A key to the abbreviations is shown below.

Key

ICTL = Integrated Transaction Services Control Server
IS = Integrated Server
ISCF = Integrated Server Control File
PATH = Pathway Interface object
PCTL = Process Control object
XPN = XPNET Interface object
TRC = Trace object


>Start Trace
1
>Stop Trace
7
2
Pathway
8
Pathway
ICTL Server Process
3
ICTL Server Process
9
ISCF
ISCF
File Read
File I/O Completion
File Read
File I/O Completion
IS Process
IS Process
XPN
4
XPN
10
5
PCTL
6
TRC
11
PCTL
12
TRC
Object X
Object Y
Object X
Object Y
Trace File

Processing involves the following steps:

1. An operator initiates a process trace at a network terminal.
2. Pathway delivers the START TRACE command to the Integrated Transaction Services Control Server process.
3. The Integrated Transaction Services Control Server process writes the START TRACE command to the appropriate Integrated Server Control File (ISCF).
4. An Integrated Server process within the designated server class receives a file I/O completion due to a change in the end-of-file on its ISCF.

   The XPNET Interface object receives the file I/O completion and sends the File I/O Completion member function to the Process Control object.
5. The Process Control object receives the member function, reads the ISCF, obtains the entered START TRACE command, and passes the command on to the Trace object for execution.
6. The Trace object receives the START TRACE command, opens a trace file and begins writing trace information to the file. It then informs the appropriate objects to begin tracing. When an object is in trace mode, it sends Trace Add member functions to the Trace object and the Trace object adds this data to the trace file.
7. A network operator enters a STOP TRACE command at a network terminal.
8. Pathway delivers the STOP TRACE command to the Integrated Transaction Services Control Server process.
9. The Integrated Transaction Services Server process writes the STOP TRACE command to the appropriate ISCF.
10. An Integrated Server process within the designated server class receives a file I/O completion due to a change in the end-of-file on its ISCF.

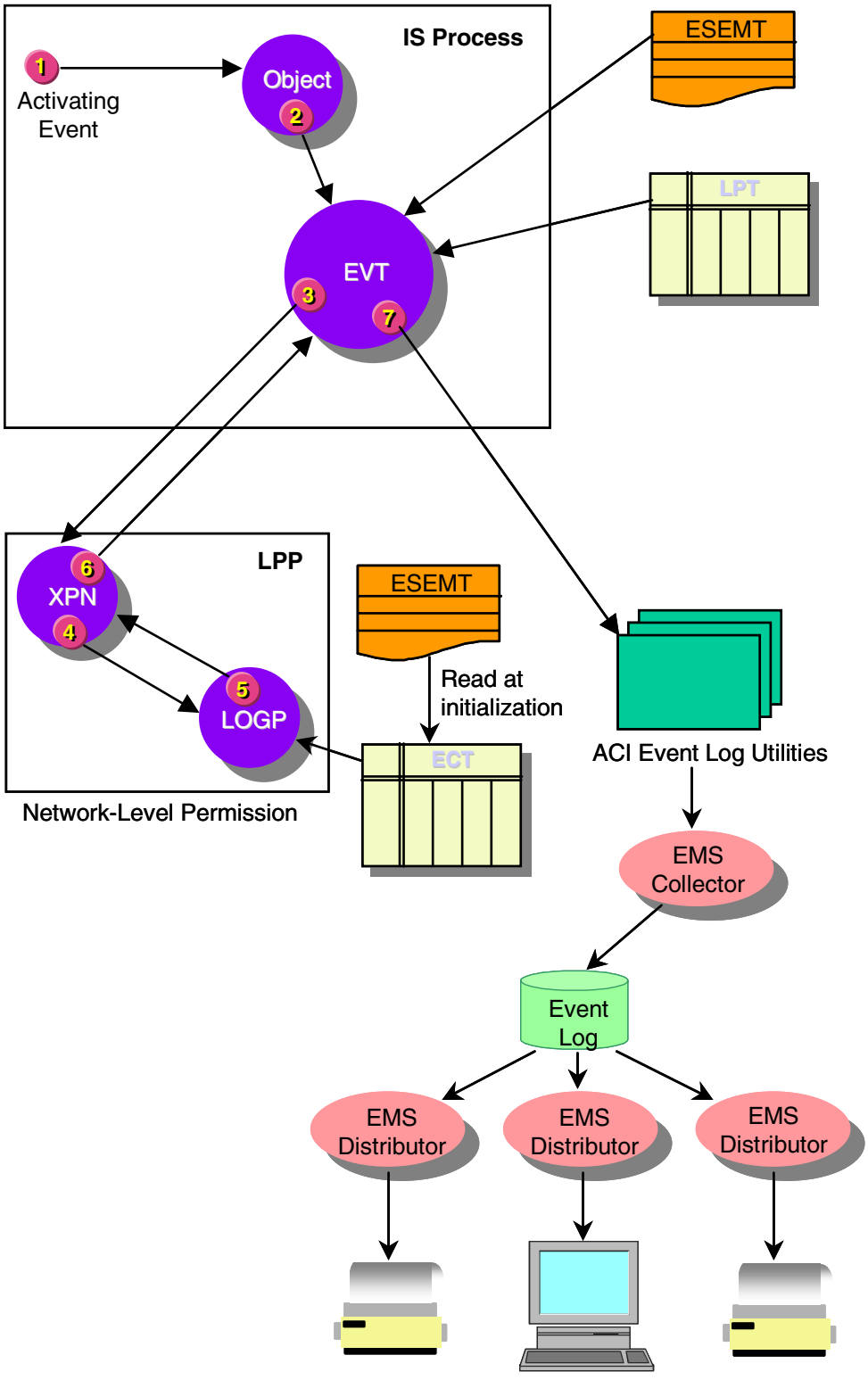
    The XPNET Interface object receives the file I/O completion and sends the File I/O Completion member function to the Process Control object.
11. The Process Control object receives the member function, reads the ISCF, obtains the entered STOP TRACE command, and passes the command on to the Trace object for execution.
12. The Trace object receives the STOP TRACE command, stops writing trace information to the trace file and closes the trace file. It also informs all objects involved in the trace to stop (disable) tracing.

Event Log Request Flow

When an object in an Integrated Server process logs an event, the Event object, and possibly the Log Permission object, determine whether the event is logged or suppressed. The Event object logs the event if it is not suppressed. The Log Permission object is used only if the event is suppressed at the network level rather than at the process level. This example flow assumes that the event generated is suppressed at the network level and that the event suppression method is by number of events. Event log processing is illustrated and described on the following pages. A key to the abbreviations is shown below.

Key

ECT = Event Control Table
EMS = Event Management System
ESEMT = Event Suppression File extended memory table
EVT = Event object
IS = Integrated Server
LPT = Log Permission Process Table
LPP = Log Permission Process
LOGP = Log Permission object
XPN = XPNET Interface object


IS Process
1
Activating Event
Object
2
ESEMT
LPT
EVT
3
7
LPP
6
XPN
4
5
LOGP
Network-Level Permission
ESEMT
Read at initialization
ECT
ACI Event Log Utilities
EMS Collector
Event Log
EMS Distributor
EMS Distributor
EMS Distributor

Processing involves the following steps:

1. An event is generated in an Integrated Server process.
2. The object in which the event occurred sends the Log member function to the Event object.
3. Using the Event Suppression File extended memory table and the Log Permission Process Table (LPT), the Event object determines that the event must be sent to a Log Permission process for network-level suppression. The Event object uses a writeread procedure to send an Event Log member function to the appropriate Log Permission process.
4. The XPNET Interface object in the Log Permission process receives the Event Log member function and forwards it to the Log Permission object.
5. The Log Permission object determines whether to suppress the event using the Event Control Table (ECT), and sends an appropriate response back to the XPNET Interface object.
6. The XPNET Interface object forwards the response to the Event object in the process where the event originated.
7. Based on the response, the Event object either logs the event or suppresses it. If the event is logged, the object uses the ACI event logging utilities to send the event to the Tandem Event Management Service (EMS) for collection and distribution.

Application Initialization Flow

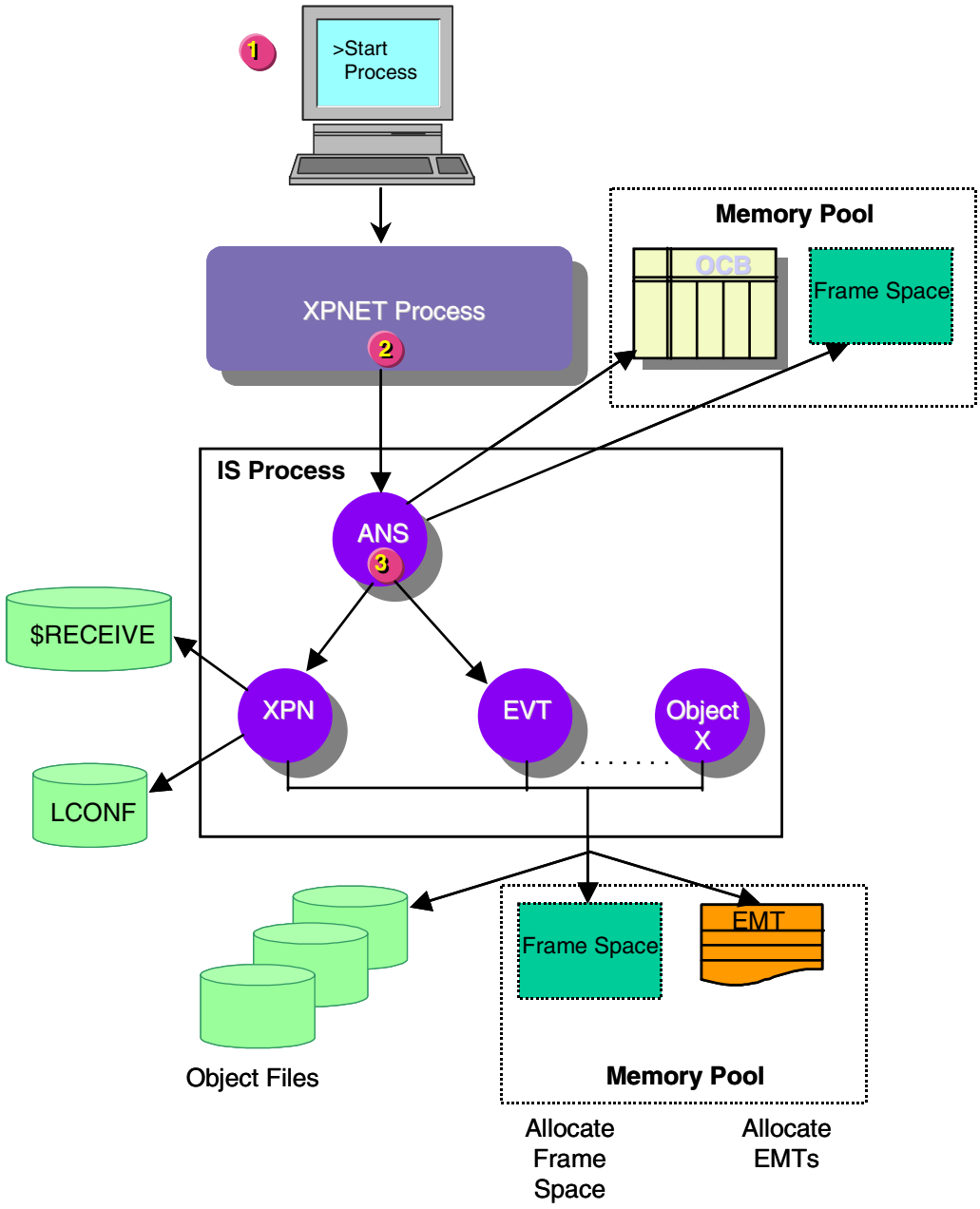
When an Integrated Server process is started, the Application Network Services object is responsible for initializing all objects in the process. The Application Network Services object always contains the main procedure for an ITS process.

During initialization processing, the object initializes itself, initializes the Event object with a default configuration (this allows events to be logged while the XPNET Interface object initializes), initializes the XPNET Interface object, and then performs phase 1 initialization of the Event object. After these objects are initialized, the Application Network Services object performs phase 1 initialization for the rest of the objects in the process. Once phase 1 initialization is complete for all objects in the process, the object performs phase 2 initialization for any objects that requested notification during phase 1.

During initialization processing, objects are registered, frame space is acquired, extended memory tables are allocated, Logical Network Configuration File (LCONF) assigns and params are read, and files are opened. Objects that request it are also informed when other objects have been initialized. Initialization processing is illustrated and described on the following page. A key to the abbreviations is shown below.

Key

ANS	=	Application Network Services object
EMT	=	Extended Memory Table
EVT	=	Event object
IS	=	Integrated Server
LCONF	=	Logical Network Configuration File
OCB	=	Object Control Block Table
XPN	=	XPNET Interface object


1
>Start
Process
Memory Pool
XPNET Process
2
OCB
Frame Space
IS Process
ANS
3
$RECEIVE
XPN
EVT
Object
X
LCONF
Frame Space
EMT
Object Files
Memory Pool
Allocate
Frame
Space
Allocate
EMTs

Processing involves the following steps:

1. An operator starts an Integrated Server process. For detailed startup procedures, refer to the ***NET24-XPNET Network Control Operations Guide***.
2. The XPNET process sends a STARTUP command to the Integrated Server process.
3. The object containing the main procedure for the Integrated Server process—the Application Network Services object—receives the command and performs the following functions to initialize and start the Integrated Server process.
    a. Performs the following to initialize the object environment:
        1) Determines whether the large-memory model or the small-memory model is used and allocates the memory pool.
        2) Initializes the Object Control Block Table (OCB).
    b. Initializes the Event object using a default configuration. This allows any events generated during XPNET Interface object initialization to be logged.
    c. Initializes the XPNET Interface object. The XPNET Interface object performs the following:
        1) Allocates its required frame space from a flat segment (large-memory model) or from the upper 32K of the stack (small-memory model).
        2) Initializes its frame, including the input and output message buffers.
        3) Registers with the Application Network Services object.
        4) Opens the $RECEIVE file.
        5) Opens the Logical Network Configuration File (LCONF) and reads its assigns and params.
        6) Allocates the source symbolic map extended memory table.
        7) Determines the the symbolic name of the process.

d. Sends the Initialize member function to all other objects in the process, beginning with the Event object. The Initialize function indicates whether the large-memory model or small-memory model is used. Each object performs the following:

   Note: Steps 1 through 3 below do not apply to the Event object, since these steps are performed when the Event object is initialized with a default configuration in step 3b.

   1) Allocates its required frame space from a flat segment (large-memory model) or from the upper 32K of the stack (small-memory model).

   2) Initializes its primary frame and any secondary frames it uses.

   3) Registers with the Application Network Services object.

   4) Reads the LCONF to obtain its assigns and params.

   5) Opens its needed files and allocates any extended memory tables.

   6) Responds to the Initialize member function.

e. Sends a Notify member function to all objects requesting notification for phase 2 initialization. Each object performs any required communication with other objects or takes appropriate actions.

f. Calls the XPNET Interface object to wait for messages from the $RECEIVE file or from any other file.

   After a message is processed, the process simply waits to receive another message.

Glossary

This glossary explains terms and acronyms commonly used in reference to the integrated transaction services (ITS) kernel objects.

Application Network Services object — Provides networking capabilities fundamental to the function of a process in the ITS environment.

Class — A set of objects sharing a common structure and behavior. A class acts as a template for an object. The ITS environment supports three classes of objects—kernel, integrated endpoint, and core.

Class can also refer to a grouping of processes or servers that perform similar functions. A server class is a group of processes that perform similar functions. Any process associated with a particular server class can service a transaction sent to that server class. An endpoint class is a group of entities through which messages are sent or received. An endpoint class is physically represented by a group of processes or stations on the network. An endpoint is logically represented by an endpoint class.

Class object — Determines the object ID associated with a specific source class value for routing purposes. This is referred to as object ID retrieval.

ECT — See Event Control Table (ECT).

EMT — See Extended Memory Table File (EMT).

Event Control Table (ECT) — A table of data from the Event Suppression File extended memory table that the Log Permission object uses to determine network-level suppression.

Event object — Handles the logging of event messages to the Tandem Event Management Service (EMS) and process-level event suppression.

Event suppression — A means of reducing the number of times a log message is generated. Event suppression is helpful when multiple processes report the same error condition or when the same error is continually reported by the same process.

Event Suppression File (ESF) extended memory table — A table of data accessed in memory from the Event Suppression File (ESF).

Extended memory table — File or table data accessed in processor memory rather than from disk. If the table data is from a file or table, the file or table data is maintained on disk as well as in memory. Memory access allows for faster data access than disk access.

Extended Memory Table File (EMT) — A file defining extended memory tables that are built by the Extended Memory Table Build utility or can be reallocated using the EMT WARMBOOT command.

Frame — A data structure in memory containing information that must always be present for an object to function. The frame is used in place of global data. The content and structure of the frame is object dependent. Data stored in a frame can include such items as file names and numbers, extended memory table segment IDs, and pointers to any dynamically allocated private data areas. It will usually contain an object's status, trace levels, extended memory table segment IDs, file names and numbers, and pointers to any other dynamically allocated data areas.

Integrated Authorization Server — Any application authorization process running in the ITS environment that is composed from kernel and core objects.

Integrated Endpoint — A source from which integrated transaction services (ITS) messages originate. An integrated endpoint is physically represented by a process or station in the network.

Integrated Endpoint process — A satellite process to the XPNET process that is composed from kernel and integrated endpoint objects and is responsible for message routing, maintaining transaction context, Measure counter manipulation, and network overload management for a particular endpoint. An Integrated Endpoint process always contains an endpoint-specific Endpoint Class Handler object.

Integrated Server — Any process running in the integrated transaction services (ITS) environment that is composed from objects. An Integrated Server is composed of the following kernel objects as well as integrated endpoint or application-specific objects: Application Network Services, Class, Event, Trace, and XPNET Interface object. An Integrated Server provides business transaction services, such as card management, authorization, external interface management, and transaction logging.

Integrated transaction services (ITS) — The ACI framework for designing, developing, configuring, and controlling processes utilizing an object-oriented design philosophy.

Integrated Server Control File (ISCF) — The mechanism by which commands initiated by network operators are delivered to objects within an Integrated Server process and by which commands can be sent from one Integrated Server process to other Integrated Server processes. The ISCF can also be used to deliver commands to procedural processes used in the ITS environment.

All Integrated Server processes post a *Control 27* call on the ISCF, which means that whenever the end of the file changes, the file is read by the Process Control object for the server.

There is one ISCF for each ITS process server class. For server classes, the control file keeps track of which servers are available to handle requests in the server class. All processes of the same server class type look at the same ISCF. A unique ISCF can exist for each process type in the system.

ISCF — See Integrated Server Control File (ISCF).

ITS — See integrated transaction services (ITS).

Kernel — Functions as the base of all ITS applications and is composed of a set of objects providing essential services to the process, such as initialization, intraprocess communication, event logging, process management, and memory management.

Lexicon — A group of messages used to provide communication between objects. For objects to be able to converse with each other they must share at least one of the same lexicons.

Log Permission object — Determines whether or not a particular log event is logged or suppressed at the network level. This object is present only in Log Permission processes.

Log Permission process — A process that determines whether an event is logged or suppressed at the network level. All log permission processing is encapsulated within the Log Permission object.

Object — A combination of privately held data and procedural statements that act upon that data. An object is called by sending it a message. The object interprets the message and performs the appropriate action. In simple terms, an object is a package of code that performs specific functions. Objects typically encompass specific processing functions which allows for small, easily managed units of code that can accommodate high transaction volumes.

OCB — See Object Control Block Table (OCB).

Object Control Block Table (OCB) — A table used by the Application Network Services object to track object frame space.

Object ID — An identification number for an object used internally by the ITS software. Currently, the Application Network Services object and the Space object share the same ID, since they are implemented as one object. The object IDs provided with the ACI-supplied default data should not be changed.

Object Space Table — A table used by the Application Network Services object to allocate and deallocate frame space used by objects belonging to the process.

Procedural — Software designed and implemented in the traditional ACI procedure-oriented architecture without the use of objects. Non-object oriented software.

Process control extended memory table — A table tracking all requests written to the Integrated Server Control File (ISCF). An entry is added to this table each time an ISCF Write member function is received by the Process Control object from other objects in the local process.

Process Control object — Receives messages from an Integrated Server Control File (ISCF) and forwards them to the appropriate object. The ISCF is the mechanism by which commands entered by network operators are delivered to the objects within the Integrated Server process and by which commands can be sent from one Integrated Server process to other Integrated Server processes.

Source Class Map File extended memory table — A table of data from the Source Class Map File (SCM) used to obtain the message handler object ID for an incoming message based on the source class.

Source symbolic map extended memory table — A table of data from the Network Environment File (NEF) used to derive the source class for an incoming message.

TLO — See Transport layer object (TLO).

Trace object — Provides the general trace facility available to all objects. The trace facility is called by a command from the Process Control object. The trace object writes the information being traced to a trace file or directly to a terminal.

Transport layer object (TLO) — The object used to communicate with the transport layer—XPNET process. The XPNET Interface object is the transport layer object for all Integrated Server processes that run under control of the XPNET process.

XPNET Interface object — Provides the message interface between Integrated Server processes and the outside environment. This includes network data messages sent to or received from the XPNET process, non-network system messages received from the operating system, and non-network data messages received from other Integrated Server processes using a writeread interface. All messages are received, sent, and replied to through the $RECEIVE file.

ACI Worldwide Inc.

Index

K

L

M

N

O

P

R

S

T

V

X

ACI Worldwide Inc.